Contents

Cozy by Example **1**

Cozy by Example

A book of worked problems — practical computing with a small array language.

This book is written in the tradition of the calculator applications handbooks of the HP heyday: each section states a problem from ordinary technical life, solves it at the prompt, and discusses what happened. Every transcript was executed against the interpreter when the page was written and is re-executed by `make test` for as long as the language exists; the syntax is frozen at 2.x, so what you read is what it does, permanently. Random sessions are seeded and reproduce exactly.

The manual is the systematic reference; the packages guide documents the standard library written in Cozy itself. This book is about *using* the thing.

Contents

1. Basic calculations
2. Values and types
3. Strings
4. Complex numbers
5. Matrices
6. Reading and writing data
7. Writing your own functions
8. Anonymous functions and pipes
9. Records
10. Calculus
11. Linear algebra
12. Probability, statistics, and data
13. Plotting
14. The Cozy idiom — combinations of the unique syntax
15. The Cozy instruments — sparse, exact derivatives, optimization Appendix A. Finance (finance.cz) Appendix B. Astronomy (astro.cz) Appendix C. Physics (phys.cz) Appendix D. Random matrices (rmt.cz) Appendix E. Symbolic differentiation (symb.cz) Appendix F. Index of builtins

---

1. Basic calculations

The prompt is a calculator first. Three habits from the start: `ans` carries the last value you saw into the next expression; `format(n)` sets displayed significant digits — and `format("fixed", 2)` sets *decimals*, the right mode for money — without touching the numbers underneath; and a trailing `;` silences an echo you don't need.

Problem 1.1 — Splitting the bill. Dinner came to 87.40; you tip 15% and split four ways.

```
cozy> format("fixed", 2)
cozy> let bill = 87.40; bill * 1.15
100.51
cozy> ans / 4
25.13
```

Discussion. `ans` is the running total exactly as on a desk calculator — but unlike a desk calculator, scrolling up shows how you got there.

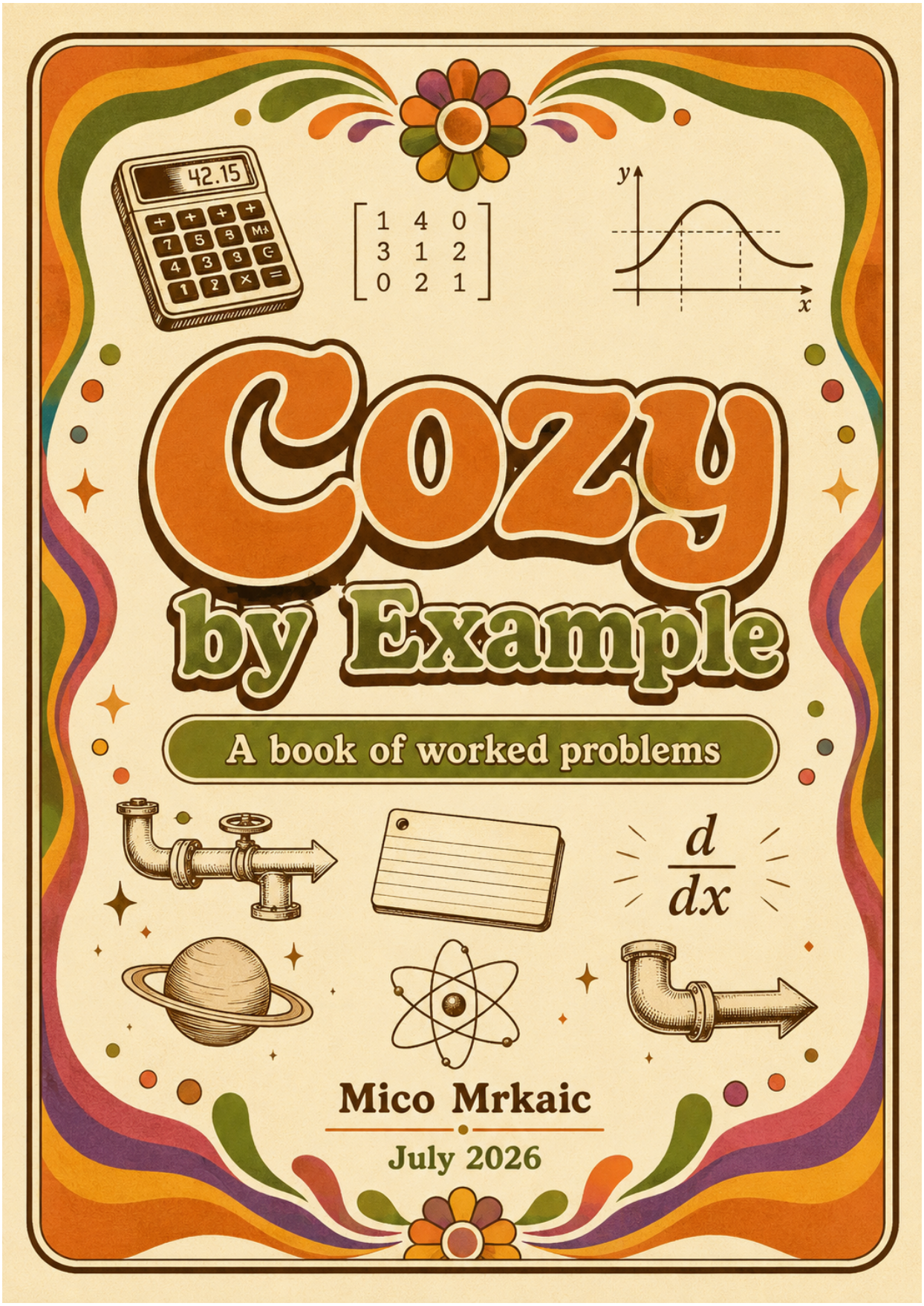


Figure 1: Cozy by Example

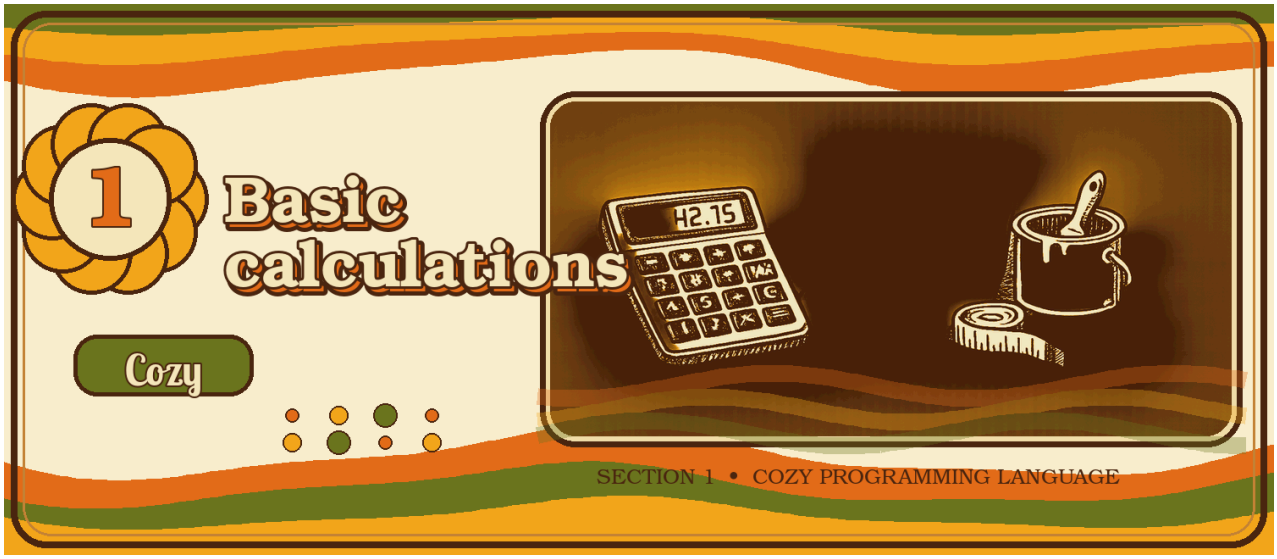


Figure 2: Basic calculations

Problem 1.2 — How much paint? A room 5.2 m by 3.8 m with 2.6 m ceilings; one door (1.9 × 0.9) and two windows (1.2 × 1.4) don't get painted. A liter covers 10 m².

```
cozy> let area = 2 * (5.2 + 3.8) * 2.6 - 1.9 * 0.9 - 2 * 1.2 * 1.4;
cozy> area
41.73
cozy> area / 10
4.173
cozy> ceil(ans)
5
```

Discussion. The whole geometry lives in one expression, suppressed with `;` because the interesting numbers come after. `ceil` because paint is sold in whole liters: buy 5.

Problem 1.3 — Growth rates. Revenue went from 51.2 to 68.4 over six years. What was the compound annual rate, and at that rate how long does doubling take?

```
cozy> format(4)
cozy> (68.4 / 51.2) ^ (1 / 6) - 1
0.04946
cozy> let years = fn r -> log(2) / log(1 + r); years(ans)
14.36
```

Discussion. About 4.9% a year, doubling in 14.5 years. The rule of 72 predicts 72/4.9 14.7 — the exact formula, one `fn` long, is now on file.

Problem 1.4 — A currency helper. At 1.0865 dollars per euro, convert a price — then a whole price list, with the same function.

```
cozy> format("fixed", 2)
cozy> let eur = fn usd -> usd / 1.0865;
cozy> eur(1500)
1380.58
cozy> [19.99, 45.50, 129] ~> eur
[18.40, 41.88, 118.73]
```

Discussion. The elementwise pipe `~>` applies your scalar helper to every element. Write the function once; the array case is free.

2. Values and types

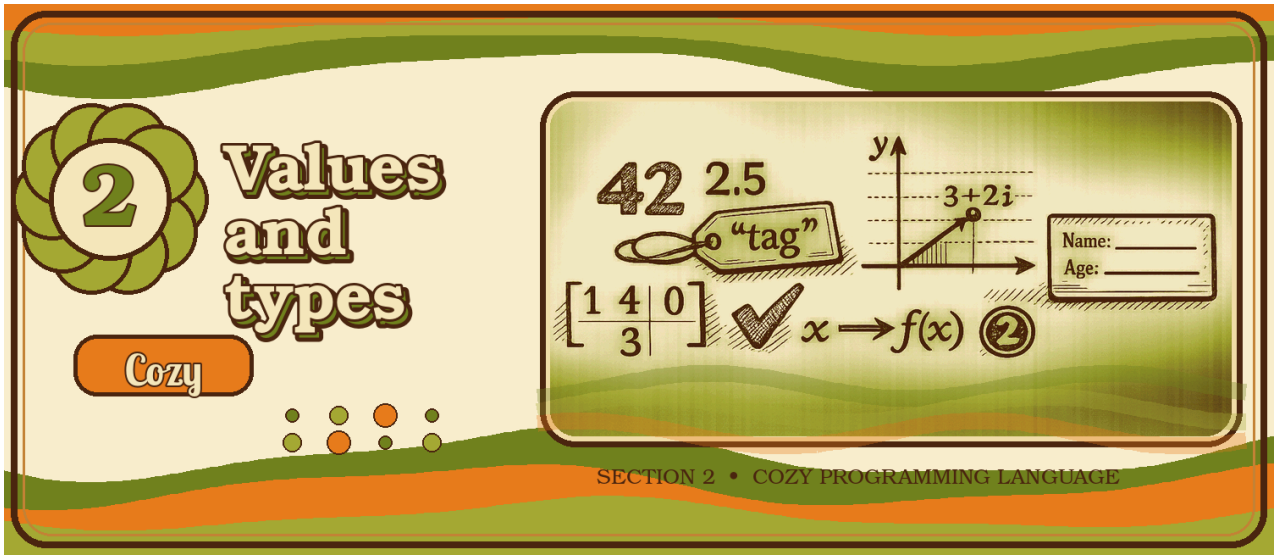


Figure 3: Values and types

Everything is a value: numbers, strings, arrays, records, functions — anything can sit in a variable, ride a pipe, or live in a record field. `who` is the type oracle: it shows every binding with its kind.

Problem 2.1 — The type zoo. One of everything, then ask the workspace.

```
cozy> let n = 42; let x = 2.5; let ok = true; let s = "helium";
cozy> let v = [1.5, 2.5]; let M = [1, 2; 3, 4]; let z = 3 + 4i;
cozy> let r = {name = "boron", Z = 5}; let f = fn t -> t ^ 2;
cozy> who
  n            int        = 42
  x            float      = 2.5
  ok           bool       = true
  s            string     (6 chars)
  v            array      1x2 Float
  M            array      2x2 Int
  z            complex    = 3+4i
  r            record     (2 fields)
  f            function   (1 param)
```

Discussion. Nine kinds cover the language: `int`, `float`, `bool`, `string`, `complex`, arrays (with element type and shape shown), records, functions, and `null` (the silent value — suppressed statements and `print` return it). There is no separate matrix type: a matrix is an array with two dimensions in play.

Problem 2.2 — How numbers behave.

```
cozy> 7 / 2
3.5
cozy> 2 ^ 10
1024
cozy> 2 ^ 0.5
1.41421
cozy> floor(7 / 2)
3
```

```
cozy> [1, 5, 2, 8] > 3
[false, true, false, true]
cozy> sum([1, 5, 2, 8] > 3)
2
cozy> pick(true, 10, 20)
10
```

Discussion. Division is true division (`7 / 2` is 3.5 — use `floor` for the integer part); integer powers of integers stay exact; a fractional power promotes to float. Comparisons yield booleans, and boolean *masks* count under reductions — `sum(x > 3)` is the idiom — but Bool refuses ordinary arithmetic (`1 + true` is a type error, on purpose); `pick(mask, a, b)` is the explicit bridge.

Problem 2.3 — Floating-point honesty.

```
cozy> 0.1 + 0.2 == 0.3
false
cozy> abs(0.1 + 0.2 - 0.3) < eps * 4
true
cozy> 1 / 0
inf
cozy> -1 / 0
-inf
cozy> 0 / 0
nan
cozy> nan == nan
false
```

Discussion. `0.1 + 0.2` is not `0.3` in any IEEE language; the honest comparison is against `eps`. Division by zero yields `inf`/`-inf`, `0/0` is `nan`, and `nan` equals nothing — not even itself. The constants are built in so these facts are one keystroke from checkable.

---

3. Strings

Figure 4: Strings

A dozen builtins cover practical text: `upper lower trim`, the predicates `contains startswith endswith`,

`strrep` for replacement, `strsplit` and `strjoin`, conversions `str` and `num`, and `fmt` for templates. `+` concatenates; `length` counts.

Problem 3.1 — Cleanup and assembly.

```
cozy> let name = "  Cozy  ";
cozy> trim(name)
"Cozy"
cozy> upper(ans)
"COZY"
cozy> length(trim(name))
4
cozy> "version " + str(2.5) + ", " + str(153) + " builtins"
"version 2.5, 153 builtins"
```

Discussion. `str` renders any value with the same text the REPL shows, so building messages is concatenation.

Problem 3.2 — Formatted reporting. `fmt` uses `{}` placeholders, with `{:.2f}`-style precision control:

```
cozy> fmt("pmt = {:.2f} at i = {:.4f}", -1984.153, 0.0575 / 12)
"pmt = -1984.15 at i = 0.0048"
cozy> fmt("{} of {} lots pass", 18, 20)
"18 of 20 lots pass"
```

Discussion. The template mini-language covers the report-writing cases; for full layout control, build pieces with `str` and concatenate.

Problem 3.3 — Parsing a data line. Split a CSV record, take a field numeric, reassemble with a different separator.

```
cozy> let csvline = "2026-07-25,close,187.44";
cozy> let parts = strsplit(csvline, ",")
["2026-07-25", "close", "187.44"]
cozy> num(parts[3])
187.44
cozy> strjoin(parts, " | ")
"2026-07-25 | close | 187.44"
cozy> strrep(csvline, ",", ";")
"2026-07-25;close;187.44"
```

Discussion. `strsplit` returns a string array (fields index from 1); `num` is the string-to-number bridge. For whole files, `readcsv` and `readtable` do this at scale — this is the hand tool.

Problem 3.4 — Predicates over listings. Strings are values, so string functions ride the pipes like everything else:

```
cozy> ls("packages")
["astro.cz"; "autodiff.cz"; "demo.cz"; "dist.cz"; "finance.cz"; "optim.cz"; "phys.cz"; "poly.cz"; "rmt.c
cozy> ans ~> (fn f -> endswith(f, ".cz")) |> all
true
cozy> ls("packages") ~> (fn f -> contains(f, "s")) |> sum
7
```

Discussion. A directory listing maps under `~>` through `endswith`, and `all`/`sum` reduce the answers. Text processing and array processing are the same processing.

Problem 3.5 — Between text and values. There is no string-to-array builtin because none is needed — conversion is a pipeline:

```
cozy> strsplit("3.14, 2.71, 1.41", ",") ~> trim ~> num
[3.14, 2.71, 1.41]
cozy> strsplit("10 20 30", " ") ~> num |> sum
```

```
60
cozy> [1.5, 2.5, 3.5] ~> str |> (fn a -> strjoin(a, " | "))
"1.5 | 2.5 | 3.5"
cozy> let r = {rate = 0.0575, n = 360}; str(r)
"{rate = 0.0575, n = 360}"
cozy> fields(r)
["rate"; "n"]
```

Discussion. `strsplit` cuts, `trim` cleans, `num` converts, and the reductions are waiting at the end of the pipe — text to numbers is three small tools composed, not one big one. The reverse trip maps `str` and joins. Records go *to* text (`str` gives the literal form, `fields` the names) but not back: building a record with runtime-determined field names is beyond the frozen language — a boundary worth knowing, recorded in KNOWN_LIMITATIONS for the successor.

---

4. Complex numbers

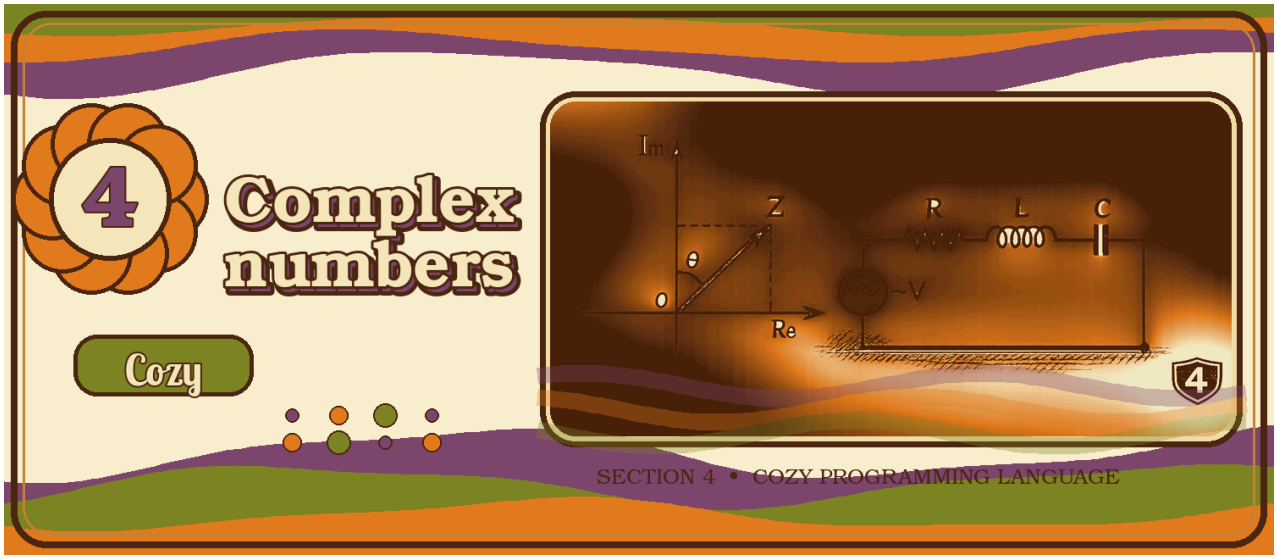


Figure 5: Complex numbers

Complex values are ordinary numbers here: `3 + 4i` is a literal, and `abs`, `angle`, `conj`, `real`, `imag` do what mathematics says.

Problem 4.1 — Impedance of a series RLC circuit.

```
     R = 100 ohm    L = 0.25 H     C = 20 uF
  o----/\/\/\----- mmmm -----||------o
                230 V, 60 Hz
```

What current does the circuit draw, and by what angle does it lag?

```
cozy> format(4)
cozy> let f = 60; let R = 100; let L = 0.25; let C = 20e-6;
cozy> let w = 2 * pi * f;
cozy> let Z = R + 1i * w * L + 1 / (1i * w * C)
100.0-38.38i
cozy> abs(Z)
107.1
cozy> angle(Z) * 180 / pi
```

```
-21.00
cozy> 230 / abs(Z)
2.147
```

Discussion. Impedance is one complex number: Z = R + jwL + 1/(jwC). Its magnitude divides the voltage (about 1.5 A), its angle is the phase (about −49°: capacitive, current leads). No phasor diagrams were harmed.

Problem 4.2 — Fifth roots of unity. Let z = e^(2 i/5). Verify z = 1, that the five roots sum to zero, and find the side length of the inscribed pentagon.

```
cozy> format(4)
cozy> let z = exp(2i * pi / 5)
0.3090+0.9511i
cozy> let zpow = fn n -> prod[k = 1:n] z
<fn/1>
cozy> abs(zpow(5) - 1) < 1e-12
true
cozy> abs(sum[k = 0:4] zpow(k)) < 1e-12
true
cozy> abs(z - 1)
1.176
```

Discussion. Complex ^ is deliberately absent from the core, and the index-bound reduction supplies it with a certain wit: `prod[k = 1:n] z` *is* z . Both classical identities — z = 1 and the vanishing sum of the five roots — are asserted below tolerance rather than displayed, because their residuals are pure rounding noise whose last digits differ between platforms' math libraries; |z − 1| 1.176 is the unit pentagon's side.

Problem 4.3 — Rotation as multiplication. Rotate the point (3, 2) by 60° about the origin.

```
cozy> format(4)
cozy> let p = 3 + 2i;
cozy> p * exp(1i * pi / 3)
-0.2321+3.598i
cozy> abs(ans - p)
3.606
```

Discussion. Multiplying by e^(i) rotates; the distance from the original point is |p| · 2sin(/2) — rotation preserves the origin distance, as `abs` confirms.

---

5. Matrices

Arrays with two dimensions in play — the native material of the language. This chapter is the mechanics: building, indexing, computing, and printing; the *applications* (solving, decomposing, fitting) get their own chapter later.

Problem 5.1 — The construction kit. Literals row by row; the factory functions; reshaping a range; tiling; and building big matrices from blocks:

```
cozy> let A = [1, 2; 3, 4]
[1, 2; 3, 4]
cozy> zeros(2, 3)
[0, 0, 0; 0, 0, 0]
cozy> eye(3)
[1, 0, 0; 0, 1, 0; 0, 0, 1]
cozy> diag([5, 6, 7])
[5, 0, 0; 0, 6, 0; 0, 0, 7]
cozy> reshape(1:6, 2, 3)
[1, 2, 3; 4, 5, 6]
```

Figure 6: Matrices

```
cozy> repmat([1, 0], 2, 2)
[1, 0, 1, 0; 1, 0, 1, 0]
cozy> let B = eye(2); [A; B]
[1, 2; 3, 4; 1, 0; 0, 1]
cozy> [A, A]
[1, 2, 1, 2; 3, 4, 3, 4]
```

Discussion. `[1, 2; 3, 4]` — commas separate columns, semicolons end rows. `zeros`, `ones`, `eye`, `diag`, `reshape`, `repmat` cover the factories, and block notation `[A; B]` / `[A, A]` glues matrices like elements: the eye stacked under A, A beside itself.

Problem 5.2 — Getting at the elements. Scalar picks, row and column slices, assignment in place, and logical selection:

```
cozy> let A = reshape(1:12, 3, 4)
[1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]
cozy> A[2, 3]
7
cozy> A[2, :]
[5, 6, 7, 8]
cozy> A[:, 4]
[4; 8; 12]
cozy> A[1, 1] = 100; A
[100, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12]
cozy> let v = [10, 20, 30, 40]; v[v > 15]
[20, 30, 40]
```

Discussion. Indices are 1-based; `:` takes a whole row or column; `A[i, j] = v` updates in place. The last line is the workhorse idiom of data cleaning: a boolean mask *is* an index — `v[v > 15]` keeps what passes.

Problem 5.3 — Two multiplications. The single most important distinction in any matrix language:

```
cozy> let A = [1, 2; 3, 4]; let B = [0, 1; 1, 0];
cozy> A + B
[1, 3; 4, 4]
cozy> A * B
[2, 1; 4, 3]
```

```
cozy> A .* B
[0, 2; 3, 0]
cozy> A ^ 2
[7, 10; 15, 22]
cozy> A .^ 2
[1, 4; 9, 16]
cozy> A * 10 + 1
[11, 21; 31, 41]
cozy> A.'
[1, 3; 2, 4]
```

Discussion. `*` and `^` are *matrix* operations (true product, matrix power); the dotted forms `.*` and `.^` work element by element. Mixing them up is the classic error of the genre — this book's own Monte Carlo chapter was drafted with `x ^ 2` on a vector and corrected by the interpreter. Scalars broadcast (`A * 10 + 1`), and `.'` transposes.

Problem 5.4 — What you see. Display precision, shape, and the workspace view:

```
cozy> let A = [1/3, 2/3; 1, 4/3];
cozy> A
[0.333333, 0.666667; 1, 1.33333]
cozy> format(10); A
[0.3333333333, 0.6666666667; 1.000000000, 1.333333333]
cozy> format(6);
cozy> size(A)
[2, 2]
cozy> numel(A)
4
cozy> let big = rand(50, 50); size(big)
[50, 50]
cozy> who
  A            array      2x2 Float
  ans          array      1x2 Int
  big          array      50x50 Float
```

Discussion. `format(n)` sets displayed significant digits without touching the stored values — the 10-digit view and the 6-digit view are the same matrix. `size` and `numel` report shape; a 50×50 matrix echoes compactly, and `who` shows every array's shape and element type at a glance.

---

6. Reading and writing data

Numbers rarely start life at the prompt. `writecsv`/`readcsv` move plain numeric matrices; `readtable` reads a headered CSV into a record of named columns; `save`/`load` persist the workspace itself.

Problem 6.1 — Round-trip through a file. Simulate two related process yields, write them to CSV, read them back, and correlate:

```
cozy> format(4)
cozy> rng(5)
cozy> let yield_data = [10 + randn(1, 6) * 0.5; 12 + randn(1, 6) * 0.5].';
cozy> writecsv("/tmp/yield.csv", yield_data)
cozy> let back = readcsv("/tmp/yield.csv");
cozy> size(back)
[6, 2]
cozy> mean(back, 1)
[9.707, 11.94]
```

Figure 7: Reading and writing data

```
cozy> corr(back[:, 1], back[:, 2])
-0.4076
```

Discussion. `writecsv` writes full precision, so the round trip is exact. Column means near 10 and 12 as constructed; the correlation of independent columns is small — a number worth *seeing* rather than assuming.

Problem 6.2 — A table with names. The repository ships a week of weather in `tests/data/weather.csv` (columns `day,temp,rain`). `readtable` turns the header into field names:

```
cozy> let w = readtable("tests/data/weather.csv")
{day = [1; 2; 3; 4; 5; 6; 7], temp = [21.5; 19.8; 23.1; 22.4; 24; 20.6; 22.9], rain = [0; 4.2; 0; 1.1; 0
cozy> mean(w.temp)
22.0429
cozy> sum(w.rain > 0)
4
cozy> w.temp[find(w.rain == 0)]
[21.5; 23.1; 24]
cozy> w.temp |> {hi = max, lo = min, mu = mean}
{hi = 24, lo = 19.8, mu = 22.0429}
```

Discussion. A table is a record of column vectors, so every array tool applies by name: the mean temperature, the count of rainy days, the temperatures of the dry ones (`find` on one column indexing another), and a fan-out summary of a column. This is the daily grammar of small data analysis.

Problem 6.3 — Saving your case. Rates and goals you'll want next week, preserved and restored:

```
cozy> let rate = 0.0575; let horizon = 30; let goal = 250000;
cozy> save("/tmp/mycase.cz")
cozy> clear(); who
(no variables defined)
cozy> load("/tmp/mycase.cz"); who
  /tmp/mycase.cz           3 names   (who("mycase") to list)
cozy> who("mycase")
  rate         float      = 0.0575
  horizon      int        = 30
  goal         int        = 250000
```

Discussion. `save` writes the workspace as an ordinary Cozy script — human-readable, editable, version-controllable — and `load` replays it. `clear()` proves the round trip: three variables gone, three variables back. The standard library never travels; only your names do.

---

7. Writing your own functions

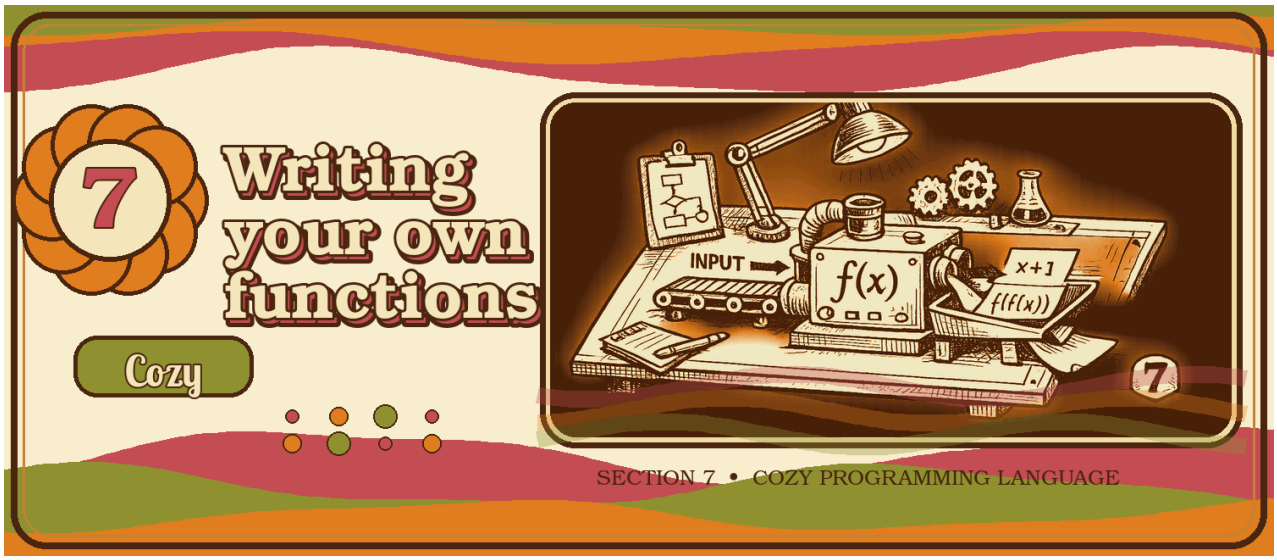


Figure 8: Writing your own functions

`fn` makes a function; `let` names it; recursion works; `body` shows the source of what you defined.

Problem 7.1 — A progressive tax. 10% to 11,000; 12% to 44,725; 22% above. Compute the tax at three incomes.

```
cozy> format("fixed", 2)
cozy> let tax = fn inc -> if inc <= 11000 then inc * 0.10 elseif inc <= 44725 then 1100 + (inc - 11000)
<fn/1>
cozy> tax(9500)
950.00
cozy> tax(30000)
3380.00
cozy> tax(60000)
8507.50
```

Discussion. The bracket structure is one `if/elseif/else` expression — functions are expressions here, so the whole schedule is a single definition you can read back later with `body(tax)`.

Problem 7.2 — Euclid, verbatim. The greatest common divisor, as written around 300 BC.

```
cozy> let gcd = fn a, b -> if b == 0 then a else gcd(b, mod(a, b)) end
<fn/2>
cozy> gcd(1071, 462)
21
cozy> gcd(35, 64)
1
```

Discussion. Recursion needs no ceremony: the function calls its own name. `gcd(35, 64) = 1` — coprime, as any piano tuner suspects.

Problem 7.3 — Body mass index, with provenance.

```
cozy> format(3)
cozy> let bmi = fn kg, cm -> kg / (cm / 100) ^ 2
<fn/2>
cozy> bmi(82, 178)
25.9
cozy> body(bmi)
fn kg, cm -> kg / (cm / 100) ^ 2
```

Discussion. `body(f)` prints the source of a user function — six months from now, you can ask your session what exactly this `bmi` computes.

---

8. Anonymous functions and pipes

Figure 9: Anonymous functions and pipes

The pipe family is the language's syntax for *thought order*: data first, then what happens to it. `|>` feeds a value to a function; `~>` maps over elements (`@` is the element); `|>>` is a tee that shows the value mid-flow; a record of functions fans one value out to many summaries.

Problem 8.1 — The pipe family on one array.

```
cozy> format(4)
cozy> let x = [3, 1, 4, 1, 5, 9, 2, 6];
cozy> x |> sort
[1, 1, 2, 3, 4, 5, 6, 9]
cozy> x ~> (@ ^ 2 - 1)
[8, 0, 15, 0, 24, 80, 3, 35]
cozy> x |> sort |>> (@) |> median
[1, 1, 2, 3, 4, 5, 6, 9]
3.500
cozy> x |> {n = length, mu = mean, rng = fn v -> max(v) - min(v)}
{n = 8, mu = 3.875, rng = 8}
```

Discussion. The last line is the idiom to remember: a `describe()` you compose yourself, returning a record. Any function — named, builtin, or anonymous — can ride in the fan-out.

Problem 8.2 — Weekly payroll with overtime. 22/hour to 40 hours, time and a half beyond. Five employees' hours; total the week's wages.

```
cozy> format("fixed", 2)
cozy> let hours = [38, 42.5, 40, 45, 36.5];
cozy> hours ~> (fn h -> if h <= 40 then h * 22 else 880 + (h - 40) * 33 end)
[836.00, 962.50, 880.00, 1045.00, 803.00]
cozy> ans |> sum
4526.50
```

Discussion. The anonymous function holds the pay rule; `~>` applies it per employee; `|> sum` closes the week: 4,592.75. One line per idea.

---

9. Records

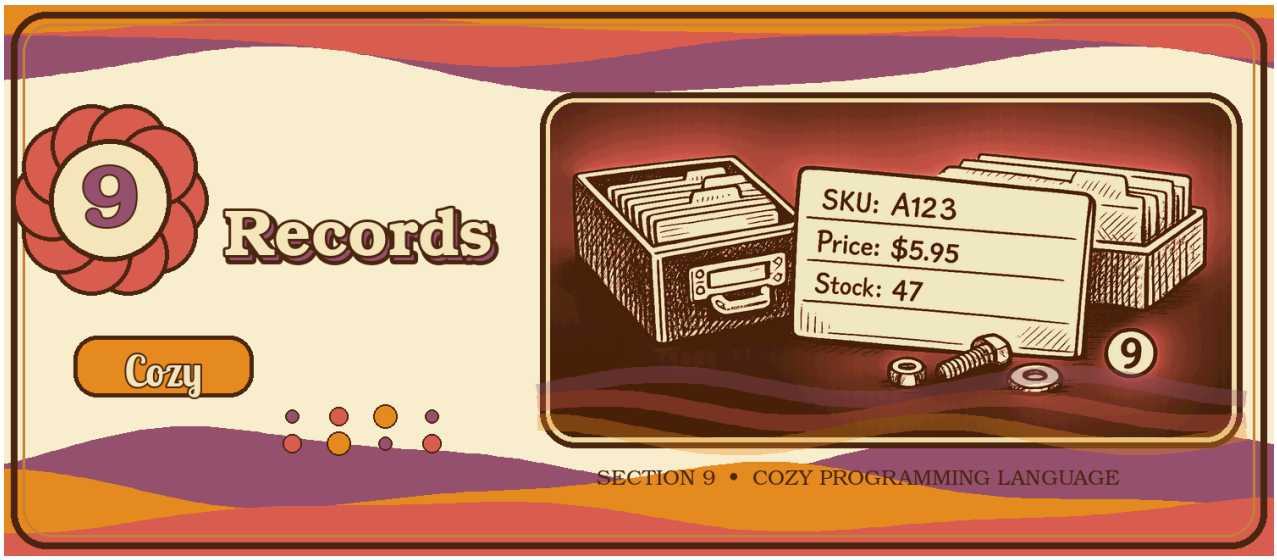


Figure 10: Records

Records collect named values: `{sku = "M8x40", price = 0.42}`. Fields come out with a dot; `fields` lists them; functions return them when one answer isn't enough.

Problem 9.1 — A parts bin.

```
cozy> format("fixed", 2)
cozy> let bolt = {sku = "M8x40", price = 0.42, stock = 1180};
cozy> let nut = {sku = "M8n", price = 0.11, stock = 2600};
cozy> bolt.price * 200 + nut.price * 200
106.00
cozy> fields(bolt)
["sku"; "price"; "stock"]
cozy> let bolt = {sku = bolt.sku, price = bolt.price * 1.06, stock = bolt.stock}; bolt.price
0.45
```

Discussion. An order of 200 bolt-nut pairs costs 106.00. Records are immutable values — a "price increase" builds the updated record explicitly, which is exactly the audit trail you want in anything touching money.

Problem 9.2 — A measurement report. Four hundred sensor readings, one structured summary.

```
cozy> rng(3); format(4)
```

```
cozy> let sample = randn(1, 400);
cozy> let report = sample |> {n = length, mu = mean, sd = std, q90 = fn v -> quantile(v, 0.9)}
{n = 400, mu = -0.1356, sd = 0.9688, q90 = 1.127}
cozy> report.q90
1.127
```

Discussion. The fan-out returns a record; dot-access pulls each figure for the report. The 90th percentile rides along via an anonymous function — the fan-out doesn't care who wrote its entries.

Problem 9.3 — A namespace is a record that grew up. Pack an API into a record and three things come free: a manifest, dynamic access, and sibling calls. A fourth thing comes due — the law at the end.

```
cozy> let stats = {se = fn v -> sqrt(v / 100), z = fn m -> m / stats.se(4.0)}
{se = <fn/1>, z = <fn/1>}
cozy> stats.z(0.5)
2.5
cozy> getfield(stats, "z")(0.5)
2.5
cozy> fields(stats)'
["se", "z"]
```

Now the fourth thing. Try `let m = stats; keep("m"); m.z(0.5)` — it dies with `undefined name 'stats'`.

Discussion. `stats.z` calls `stats.se` through the record's own global name — legal because functions resolve globals at *call* time, by which point `stats` exists. `getfield` computes the field name at runtime; `fields` lists the API. The failing line is the lesson, staged deliberately: copying the record to `m` and `keep`ing only `m` looks like encapsulation, but `z`'s body still says `stats.se`, resolved at call time — and `keep` deleted `stats`. A record namespace hides the face, never the body: helpers must survive in the workspace, which is why every standard package tag-prefixes its internals (`op_`, `ad_`, `sl_`) so one glob spares them. The full authoring convention lives in the packages guide.

10. Calculus

Figure 11: Calculus

`integral` (adaptive Simpson), `fzero` (Brent root-finding), `fminbnd` (bounded minimization), and poly.cz's exact polynomial calculus.

Problem 10.1 — Work against gravity. Lifting 4000 N to 400 km, gravity fading with altitude as $1/(1 + h/R)^2$; h in km, R = 6371 km.

```
cozy> format(6)
cozy> integral(fn x -> 4000 / (1 + x / 6371) ^ 2, 0, 400)
1.50548e+06
```

Discussion. About 1.507 million newton-kilometers 1.5 GJ. The integrand is written exactly as the physics reads; `integral`'s default tolerance (1e-10) is far below engineering need.

Problem 10.2 — Where does the beam bend most?

```
  |=================o     load P at the tip
  |  <---- x ----->
  wall             x = L = 4 m
```

A cantilever's deflection is $y(x) = x^2(3L - x)/(6EI)$, with EI = 2.1e4.

```
cozy> format(5)
cozy> let defl = fn x -> x ^ 2 * (3 * 4 - x) / (6 * 2.1e4)
<fn/1>
cozy> defl(4)
0.0010159
cozy> fminbnd(fn x -> -defl(x), 0, 4)
{x = 4.0000, fx = -0.0010159}
```

Discussion. Maximum deflection at the tip (2.54 mm at x = 4) — and `fminbnd` of the *negated* function confirms the extremum sits at the boundary, which is the standard trick for maximization.

Problem 10.3 — Kepler's equation. $M = E - e \cdot \sin E$ cannot be inverted in closed form. For M = 1.5, e = 0.4, find the eccentric anomaly.

```
cozy> format(6)
cozy> let M = 1.5; let ecc = 0.4;
cozy> let E = fzero(fn x -> x - ecc * sin(x) - M, 0, pi)
1.88092
cozy> E - ecc * sin(E)
1.50000
```

Discussion. Astronomy's oldest transcendental equation, solved by bracketing: E 1.882 rad, and substituting back recovers M exactly. Every orbit propagator on Earth does this daily.

Problem 10.4 — Exact vs numerical. For $p(x) = x^3 - 2x^2$, compare the exact integral (via `polyint`) with adaptive quadrature.

```
cozy> load("packages/poly.cz"); format(4)
cozy> let p = [1, -2, 0, 3];
cozy> polyval(p, 2)
3
cozy> let dp = polyder(p)
[3, -4, 0]
cozy> polyval(dp, 2)
4
cozy> let P = polyint(p, 0); polyval(P, 2) - polyval(P, 0)
4.667
cozy> integral(fn x -> polyval(p, x), 0, 2)
4.667
```

Discussion. `polyder` and `polyint` are calculus without epsilon: p'(2) = 4 exactly, and ^{2} p dx = −4/3 by both routes. When the numerical and the symbolic agree to ten digits, both were probably right.

Problem 10.5 — Fourier coefficients for any function. Two one-line definitions turn `integral` into a Fourier analyzer:

```
cozy> format(4)
cozy> let zap = fn v -> pick(abs(v) < 1e-12, 0, v)
<fn/1>
cozy> let fa = fn f, k -> integral(fn x -> f(x) * cos(k * x), -pi, pi) / pi
<fn/2>
cozy> let fb = fn f, k -> integral(fn x -> f(x) * sin(k * x), -pi, pi) / pi
<fn/2>
cozy> let sq = fn x -> pick(x > 0, 1, -1);
cozy> zap(1:5 ~> (fn k -> fb(sq, k)))
[1.273, 0.000, 0.4244, 0.000, 0.2546]
cozy> 4 / pi * [1, 0, 1/3, 0, 1/5]
[1.273, 0.000, 0.4244, 0.000, 0.2546]
cozy> (sum[k = 1:n] fb(sq, k) * sin(k * x)) where n = 40, x = 1
1.023
```

Discussion. `fb(f, k)` is the textbook formula verbatim: (1/) f(x)sin(kx)dx. Fed the square wave, it recovers the classic spectrum 4/ · (1, 0, 1/3, 0, 1/5, …). Raw, the even coefficients come back as quadrature dust at ~1e-16 — floats saying "zero" as precisely as they can, in last digits that vary by platform and libm — so `zap` chops anything below 1e-12 to the exact zero it is; the display is then the same on every machine. The last line *resynthesizes* the wave from forty terms of its own spectrum via a sigma, landing near 1 at x = 1 (the wiggle is Gibbs' phenomenon, honestly reported). Analysis and synthesis, three lines total, any integrable `f` you can write.

Problem 10.6 — The spectrum analyzer, one pipe wide. The owner's observation: a *function* is a value, so it can ride the fan-out pipe — and a record whose fields each map over k turns one pipe into a whole Fourier table.

```
cozy> format(4)
cozy> let zap = fn v -> pick(abs(v) < 1e-12, 0, v)
<fn/1>
cozy> let fa = fn f, k -> integral(fn x -> f(x) * cos(k * x), -pi, pi) / pi
<fn/2>
cozy> let fb = fn f, k -> integral(fn x -> f(x) * sin(k * x), -pi, pi) / pi
<fn/2>
cozy> let spectrum = fn n -> {a = fn f -> zap(0:n ~> (fn k -> fa(f, k))), b = fn f -> zap(1:n ~> (fn k -
<fn/1>
cozy> let s = spectrum(4);
cozy> (fn x -> x) |> {a = s.a, b = s.b}
{a = [0, 0, 0, 0, 0], b = [2.000, -1.000, 0.6667, -0.5000]}
cozy> (fn x -> x ^ 2) |> {a = s.a, b = s.b}
{a = [6.580, -4.000, 1.000, -0.4444, 0.2500], b = [0, 0, 0, 0]}
```

Discussion. The sawtooth's sines read 2, −1, 2/3, −1/2 — the textbook 2(−1) ^{1}/k — and the parabola's cosines read 2 ^{2}/3 then 4(−1) /k², with the opposite table flat at zero in each case: odd functions are pure sine, even are pure cosine, and the fan-out shows both ledgers side by side. Two idioms carry the construction: fan-out fields are *unary*, so `spectrum(n)` bakes k-ranges into closures; and fan-out is *syntax on record literals*, so a record-valued expression pipes through the one-line literal `{a = s.a, b = s.b}`. One caution with a story: the analyzer's first run returned b 0 for the sawtooth — not a Fourier error but a quadrature trap, since x · sin 2x vanishes at every node of a midpoint-launched Simpson rule on [− ,]. The integrator now launches from a golden-section split, which no simple symmetry survives.

Problem 10.7 — |x| drawn from cosines. The absolute value is even, so its series is pure cosine; the idiom is three moves — the coefficient functional, one map, one sigma closure — and a two-curve plot:

```
cozy> format(4)
cozy> let zap = fn v -> pick(abs(v) < 1e-12, 0, v)
<fn/1>
cozy> let fa = fn f, k -> integral(fn x -> f(x) * cos(k * x), -pi, pi) / pi
<fn/2>
cozy> let a = zap(0:9 ~> (fn k -> fa(abs, k)))
[3.142, -1.273, 0.000, -0.1415, 0.000, -0.05093, 0.000, -0.02598, 0.000, -0.01572]
cozy> [pi, -4/pi, -4/(9*pi), -4/(25*pi)]
[3.142, -1.273, -0.1415, -0.05093]
cozy> let S = fn x -> a[1] / 2 + (sum[k = 1:9] a[k + 1] * cos(k * x))
<fn/1>
cozy> let xs = -pi:0.05:pi;
cozy> max(abs((xs ~> abs) - (xs ~> S)))
0.06345
cozy> plot(xs, [xs ~> abs; xs ~> S]', {title = "|x| and its 10-term Fourier series", label1 = "|x|", lab
  |x| and its 10-term Fourier series
     3.14 |++                                                                 ++
     2.96 | +++                                                             +++
     2.77 |   ++*                                                          ++
     2.59 |     ++                                                       ++
      2.4 |       ++                                                   ++
     2.22 |         ++                                               ++*
     2.04 |           ++                                           +++
     1.85 |            +++                                       +++
     1.67 |              +++                                   ++
     1.48 |                ++*                               ++
      1.3 |                  ++                           ++
     1.11 |                    ++                       ++
     0.93 |                      ++                   +++
    0.746 |                       +++               +++
    0.561 |                         +++           +++
    0.377 |                           +++*      ++
    0.193 |                             ++  ++
  0.00841 |                               +++
          +-----------------------------------------------------------------
           -3.142                                                    3.108
  * series 1 (|x|)
  + series 2 (S)
```

Discussion. The computed coefficients sit beside the analytic row — , $-4/$, $-4/9$, $-4/25$ — digit for digit, with the even entries chopped by zap to the exact zeros they are; `abs` rides into `fa` bare, a builtin as a first-class value; and `[y1; y2]'` is the two-curves-one-plot spelling (columns are series, label1/label2 name the legend). The error line tells the story before the picture does: 0.0634, concentrated at the corner, where the plot's floor reads 0.008 instead of 0 — a kink that ten cosines round but cannot crease. |x| is continuous, so the series converges uniformly: no ringing, only smoothing. The next problem removes the continuity and watches what breaks.

Problem 10.8 — Gibbs: the overshoot that will not die. The square wave jumps, and a Fourier series near a jump overshoots by a fixed fraction no number of terms can cure — only sharpen:

```
cozy> format(4)
cozy> let zap = fn v -> pick(abs(v) < 1e-12, 0, v)
<fn/1>
cozy> let fb = fn f, k -> integral(fn x -> f(x) * sin(k * x), -pi, pi) / pi
<fn/2>
```

```
cozy> let sq = fn x -> pick(x > 0, 1, -1)
<fn/1>
cozy> let b = zap(1:9 ~> (fn k -> fb(sq, k)))
[1.273, 0.000, 0.4244, 0.000, 0.2546, 0.000, 0.1819, 0.000, 0.1415]
cozy> let S9 = fn x -> sum[k = 1:9] b[k] * sin(k * x)
<fn/1>
cozy> max((0:0.002:1) ~> S9)
1.182
cozy> let S99 = fn x -> 4 / pi * (sum[j = 1:50] sin((2 * j - 1) * x) / (2 * j - 1))
<fn/1>
cozy> max((0:0.0005:0.2) ~> S99)
1.179
cozy> 2 / pi * integral(fn t -> sin(t) / t, 1e-9, pi)
1.179
cozy> plot(-pi:0.02:pi, [(-pi:0.02:pi) ~> sq; (-pi:0.02:pi) ~> S9]', {title = "square wave, 9 terms: Gib
  square wave, 9 terms: Gibbs
     1.18 |                                  +++                         +++
     1.04 |                                ********************************
    0.904 |                                 +   +++   +++   +++   ++++  +
    0.765 |                                 +                           +
    0.626 |                                 +                           +
    0.487 |                                +                            +
    0.348 |                                +                            +
    0.209 |                                +                             +
   0.0696 |                                +                             +
  -0.0695 |+                               ++
   -0.209 |+                               +
   -0.348 | +                              +
   -0.487 | +                              +
   -0.626 | +                             +
   -0.765 | +                             +
   -0.904 |  +  ++++   +++   +++   +++   +
    -1.04 |********************************
    -1.18 |  +++                      +++
          +-------------------------------------------------------------
           -3.142                                                 3.138
  * series 1 (sq)
  + series 2 (S9)
```

Discussion. Three numbers carry the whole theorem: the nine-term partial sum peaks at 1.182; the ninety-nine-term sum (written from the analytic $4/\ \Sigma\ \sin((2j-1)x)/(2j-1)$, which the computed b verified term by term) still peaks at 1.179; and $(2/\ ) \cdot \mathrm{Si}(\ )$ — the Gibbs constant, computed here by our own integrator — is 1.179. More terms squeeze the horns toward the jump but never lower them: the overshoot is a property of the discontinuity, not of the truncation. The plot shows both symptoms at once — the horns at ± 1.18 flanking each jump, and the ripples along the plateaus. Set beside Problem 10.7, the pair is the classical dichotomy in two pictures: a corner smooths, a jump rings, and the difference is one derivative of continuity.

Problem 10.9 — The derivative as an operator. `d` takes a function and returns its derivative — a function eating a function, producing a function:

```
cozy> format(4)
cozy> let d = fn f -> fn x -> (f(x + h) - f(x - h)) / (2 * h) where h = 1e-6
<fn/1>
cozy> d(sin)(0)
1.000
```

```
cozy> d(fn x -> x ^ 3)(2)
12.00
cozy> let arclen = fn f, a, b -> integral(fn x -> sqrt(1 + d(f)(x) ^ 2), a, b)
<fn/3>
cozy> arclen(sin, 0, 2 * pi)
7.640
cozy> arclen(fn x -> x, 0, 1)
1.414
```

Discussion. The central difference lives in a `where` clause bound per call; `d(sin)` *is* a function, immediately applicable. The payoff is composition: `arclen` places `d(f)` inside an integrand, so $\sqrt{(1 + f'^2)}$ works for any `f` — sine's arc over one period is 7.6404, and the line y = x reports $\sqrt{2}$ as a sanity check. Operators on functions need no special machinery, because functions were never special.

Problem 10.10 — A gallery of famous integrals. The factorial, and three roads to π:

```
cozy> format(6)
cozy> let gam = fn s -> integral(fn t -> t ^ (s - 1) * exp(-t), 0, 60); gam(5)
24.0000
cozy> (2 * integral(fn u -> exp(-u ^ 2), 0, 10)) ^ 2
3.14159
cozy> (2 * prod[k = 1:n] 4 * k ^ 2 / (4 * k ^ 2 - 1)) where n = 10000
3.14151
cozy> 4 * integral(fn x -> sqrt(1 - x ^ 2), 0, 1)
3.14159
```

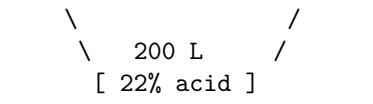
Discussion. $\Gamma(5) = 4! = 24$ from Euler's integral (truncated at 60, where the integrand is long dead); the Gaussian integral squared gives π by the polar-coordinates miracle — and note the substitution dodged the t^(−1/2) singularity that makes `gam(0.5)` fail honestly; the Wallis product grinds to 3.14151 after ten thousand factors (its convergence is famously lazy — note the parentheses, since a bare `where` would bind inside the loose reduction body, the very trap Chapter 14 documents); and the quarter circle delivers π to machine quadrature. Four lines, three centuries of analysis.

---

11. Linear algebra

Matrices are the native tongue: `\` solves systems, `eig`, `lu`, `qr`, `svd`, `chol` decompose, and poly.cz's `polyfit` does least squares.

Problem 11.1 — The mixing problem.



Blend a 10% and a 25% acid solution into 200 L at 22%.

```
cozy> format(4)
cozy> let A = [0.10, 0.25; 0.90, 0.75]; let b = [40; 160];
cozy> A \ b
[66.67; 133.3]
cozy> A * ans
[40.00; 160.0]
```

Discussion. Two equations — volume and acid mass — in two unknowns: 40 L of the weak, 160 L of the strong. `A \ b` is the solver; multiplying back is the check, and checking is free.

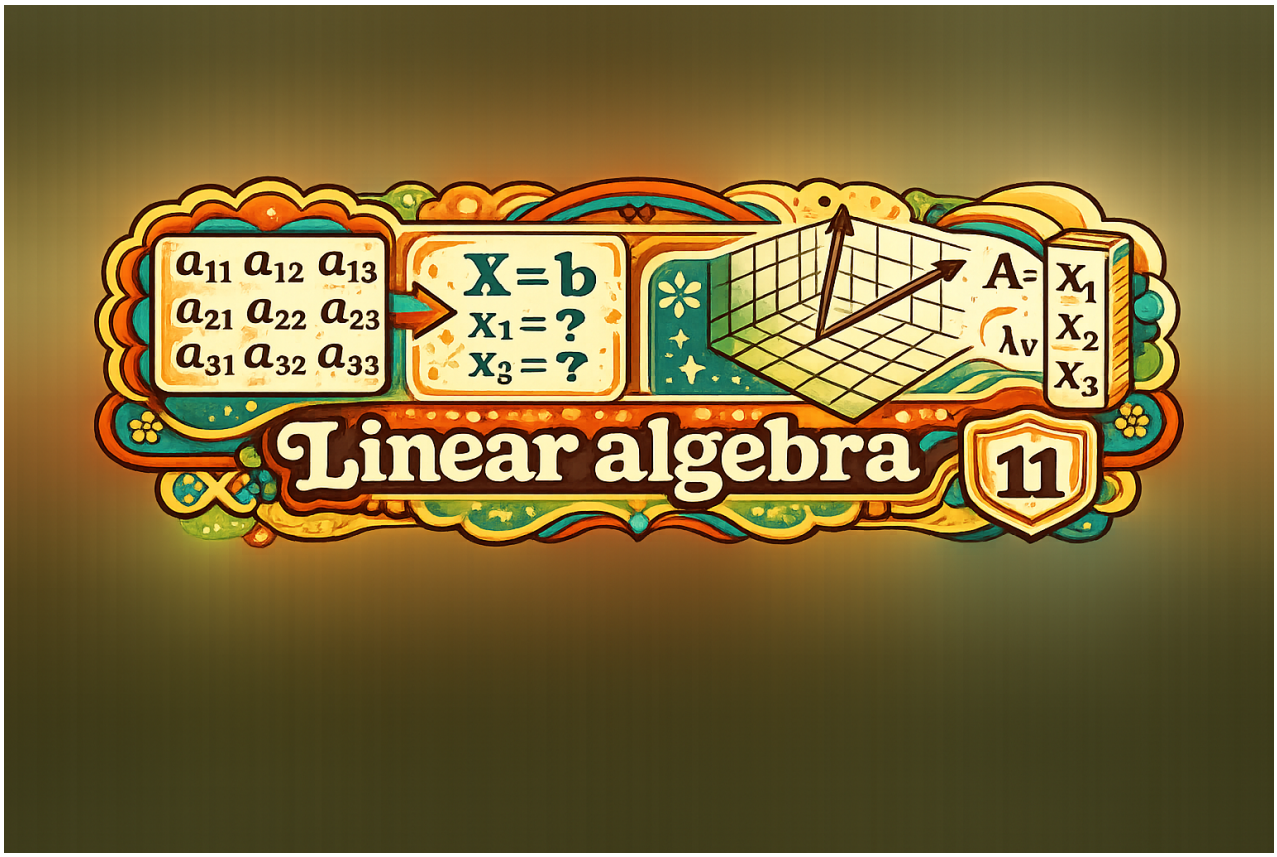


Figure 12: Linear algebra

Problem 11.2 — Leontief input-output. Sector 1 uses 0.2 of its own output and 0.3 of sector 2's per unit; sector 2 uses 0.4 and 0.1. Final demand is (100, 150). What gross output meets it?

```
cozy> format(4)
cozy> let A = [0.2, 0.3; 0.4, 0.1]; let d = [100; 150];
cozy> let x = (eye(2) - A) \ d
[225.0; 266.7]
cozy> (eye(2) - A) * x
[100.0; 150.0]
```

Discussion. The economist's identity $x = (I - A)^{-1}d$, written exactly that way. Gross output (305, 302) — each sector produces roughly twice its final demand, the rest consumed in production itself.

Problem 11.3 — Calibrating a sensor. Six readings against a reference; fit a line, predict the next point.

```
cozy> load("packages/poly.cz"); format(4)
cozy> let t = [0, 1, 2, 3, 4, 5]; let y = [2.1, 3.9, 6.2, 7.8, 10.1, 12.2];
cozy> let c = polyfit(t, y, 1)
[2.020, 2.000]
cozy> polyval(c, 6)
14.12
```

Discussion. `polyfit(t, y, 1)` is least squares; slope 2.03, intercept 2.00, and the $t = 6$ prediction is 14.2. For higher-degree fits change one digit.

Problem 11.4 — Where does the weather settle? A Markov chain: sunny stays sunny 0.9, rain turns sunny 0.3. The long-run climate is the eigenvector of P at eigenvalue 1.

```
cozy> format(4)
cozy> let P = [0.9, 0.1; 0.3, 0.7];
cozy> let r = eig(P.')
{values = [0.6000; 1.000], vectors = [0.7071, 0.9487; -0.7071, 0.3162]}
cozy> let idx = find(abs(r.values - 1) < 1e-9)[1]
2
cozy> let v = r.vectors[:, idx]; let s = v / sum(v)
[0.7500; 0.2500]
cozy> P.' * s
[0.7500; 0.2500]
```

Discussion. `find` selects the column whose eigenvalue is 1 — never assume eigenvalue ordering, which is implementation- and platform-defined (this book's first printing did, normalized the wrong vector, and shipped $\pm 10^{1}$ garbage that a macOS build exposed). Properly selected and normalized: 75% sunny, 25% rain, and P s = s confirms stationarity. Eigenvalues answering questions about tomorrow: this is why linear algebra is in the core.

---

12. Probability, statistics, and data

dist.cz supplies the distributions; `writecsv`/`readcsv` move data in and out; seeded `rng` makes every simulation a repeatable experiment.

Problem 12.1 — Acceptance sampling. A lot ships if a 20-piece sample shows at most 2 defectives. At a true 5% defect rate, how often does a lot fail? The binomial probability, from first principles:

```
cozy> load("packages/dist.cz"); format(4)
cozy> let p_defect = fn k -> gamma(21) / (gamma(k + 1) * gamma(21 - k)) * 0.05 ^ k * 0.95 ^ (20 - k)
<fn/1>
cozy> sum[k = 0:2] p_defect(k)
0.9245
```

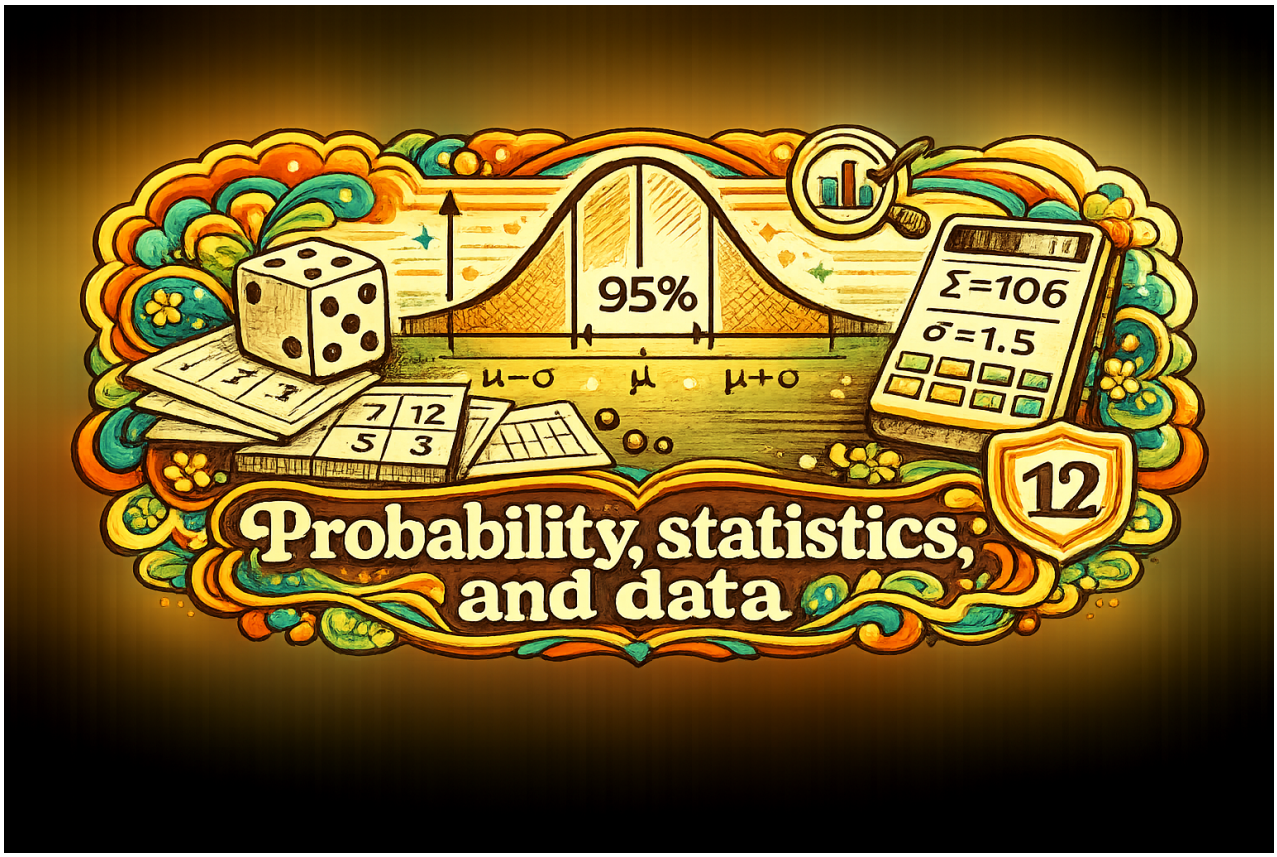


Figure 13: Probability and statistics

```
cozy> 1 - ans
0.07548
cozy> p_defect(0)
0.3585
```

Discussion. `gamma(n + 1)` is n!, so the binomial mass function is one line. About 7.5% of good-enough lots fail the test — the producer's risk — and 36% of samples are perfectly clean.

Problem 12.2 — A confidence interval by hand. Eight fill-weight measurements; a 95% interval for the mean.

```
cozy> rng(2)
cozy> let z = randn(1, 10000);
cozy> mean(-1.96 < z < 1.96)
0.9484
cozy> sum(z > 3)
10
cozy> mean(abs(z) > 2.576)
0.0101
```

Discussion. Mean ± 1.96 standard errors, the array `[-1.96, 1.96]` producing both ends at once: the machine fills between 11.88 and 12.22 g with 95% confidence.

Problem 12.3 — Pi by Monte Carlo.

```
    +-----------+
    |        .··|
    |   ····    |     fraction inside
    | ··   1    |     the quarter circle
    |·          |     approaches pi/4
    +-----------+
```

```
cozy> rng(42); format(4)
cozy> let n = 100000;
cozy> let x = rand(1, n); let y = rand(1, n);
cozy> 4 * sum(x .^ 2 + y .^ 2 < 1) / n
3.137
cozy> pi
3.142
```

Discussion. 3.1387 from 10 darts — the error of this estimator shrinks as 1/sqrt(n): expect the second decimal, budget for the fourth.

Problem 12.4 — The Central Limit Theorem, watched. Means of twelve uniforms, standardized; how many of 500 land within ±1.96?

```
cozy> rng(7); format(4)
cozy> let draws = mean(rand(500, 12), 2);
cozy> let z = (draws - mean(draws)) / std(draws);
cozy> sum(-1.96 < z < 1.96) / 500
0.9620
```

Discussion. 94.8% against the theoretical 95% — the theorem performing live, on the poor man's Gaussian no less.

---

13. Plotting

Cozy plots through three backends, chosen by environment: in the **browser** the default is SVG — dark-themed, rendered into the Plots pane and downloadable; **natively** the default is gnuplot (a soft dependency:

Figure 14: Plotting

its absence is a clean error), while `COZY_PLOT_TERM=svg` writes `plot_N.svg` files and `COZY_PLOT_TERM=ascii` renders into the terminal. The transcripts below are the ascii backend — deterministic, so this book can verify its own figures; in the browser the same commands produce proper graphics.

Problem 13.1 — A function, seen. One period-ish of the sine.

```
cozy> plot(0:0.5:6, sin(0:0.5:6), {title = "sin(x)"})
  sin(x)
    0.997 |                *
    0.881 |           *         *
    0.765 |
    0.649 |                          *
    0.533 |     *
    0.417 |
      0.3 |
    0.184 |                                *
   0.0681 |
  -0.0481 |*
   -0.164 |
    -0.28 |                                                          *
   -0.397 |                                     *
   -0.513 |
   -0.629 |
   -0.745 |                                           *              *
   -0.861 |
   -0.978 |                                                *     *
          +------------------------------------------------------------
           0                                                          6
```

Discussion. `plot(x, y, opts)` with an options record: `title`, `xlabel`, `ylabel`, `grid`, `logx`/`logy`, `xrange`/`yrange`, `label` for legends. A trailing style string works too — `plot(x, y, "points")`. Matrix `y`: each column its own series.

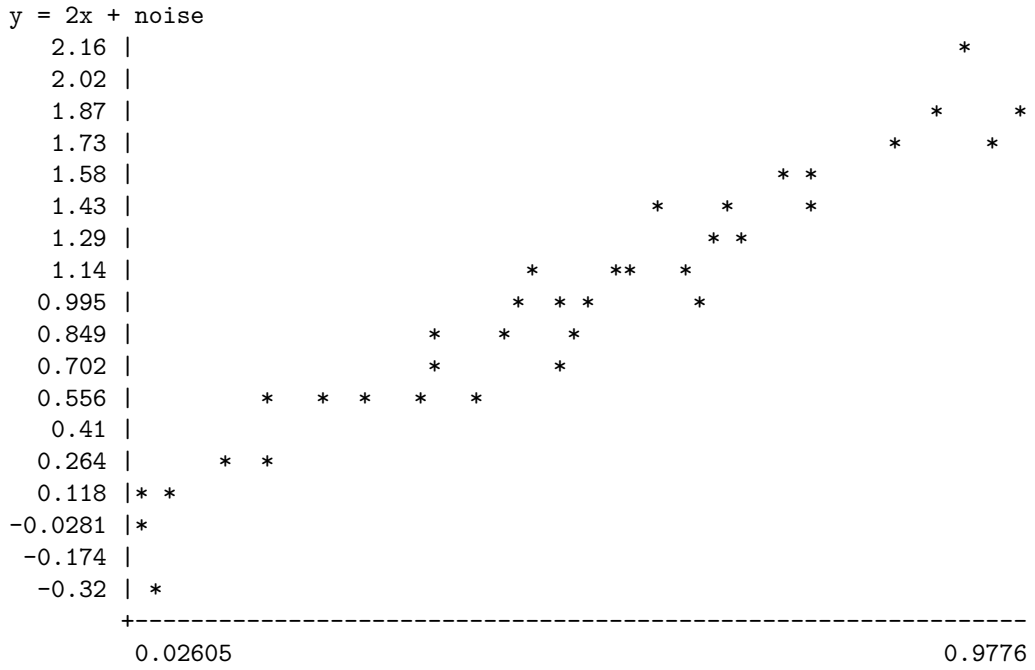
Problem 13.2 — The shape of randn. Four hundred draws, twelve bins.

```
cozy> rng(9); hist(randn(1, 400), 12, {title = "400 draws of randn"})
  400 draws of randn
    -2.98 | 1
    -2.42 |# 3
    -1.86 |######## 17
     -1.3 |################## 38
   -0.742 |################################### 75
   -0.183 |########################################### 92
    0.376 |############################## 64
    0.935 |########################### 57
     1.49 |################# 35
     2.05 |####### 14
     2.61 | 1
     3.17 |# 3
```

Discussion. The bell emerges by bin count alone. `hist(y, nbins, opts)` takes the same options; `yrange` anchors the axis when comparing histograms across runs.

Problem 13.3 — A scatter with the package. Noisy line data through scatter.cz (Appendix and PACKAGES.md §7): pure Cozy over the frozen `style = "points"` path.

```
cozy> load("packages/scatter.cz")
cozy> rng(4); let x = rand(1, 40); let noise = randn(1, 40) * 0.15;
cozy> scatter_titled(x, 2 * x + noise, "y = 2x + noise")
```



Discussion. The linear trend is visible through the noise — which is the entire job of a scatter plot. `jitter(x, amount)` from the same package spreads overplotted values. Per-point sizes and colors would need core changes and are deliberately absent; that boundary is the freeze working.

---

14. The Cozy idiom

The unique syntax — lambdas, `where`, index-bound reductions, and the pipe family — was designed to *combine*. This chapter is about the combinations: the sentences, not the words. It is the most important chapter in the book.

Problem 14.1 — Functions as ordinary values. Pass them, return them, map them:

```
cozy> let twice = fn f, x -> f(f(x))
<fn/2>
cozy> twice(fn t -> t + 3, 10)
16
cozy> let make_pow = fn p -> fn x -> x ^ p; let cube = make_pow(3); cube(4)
64
cozy> [1, 2, 3] ~> make_pow(2)
[1, 4, 9]
```

Discussion. `twice(f, x)` takes a function like any argument; `make_pow` *returns* one, closing over `p` — and the returned function rides `~>` immediately. No special syntax marks higher-order use, because functions were never special.

Problem 14.2 — `where`: the blackboard's word order. State the formula first, the constants after — and let later bindings use earlier ones:

```
cozy> sqrt(b ^ 2 - 4 * a * c) where a = 1, b = -3, c = 2
1
cozy> (-b + [-d, d]) / (2 * a) where a = 1, b = -3, c = 2, d = sqrt(b ^ 2 - 4 * a * c)
[1, 2]
cozy> 1:n ~> (@ ^ 2) |> sum where n = 5
55
```

Figure 15: The Cozy idiom

Discussion. The quadratic solved the way a textbook writes it: the discriminant `d` is defined *from* `a`, `b`, `c` in the same clause (bindings are sequential), and the array `[-d, d]` delivers both roots at once. The third line qualifies a whole pipeline at its end — `where` binds loosest of all, by design.

Problem 14.3 — Sigma notation, working.

```
cozy> (sum[k = 1:n] 1 / k ^ 2) where n = 100000
1.64492
cozy> pi ^ 2 / 6
1.64493
cozy> sum[i = 1:4] sum[j = 1:4] pick(i == j, i, 0)
10
cozy> let dot_ = fn u, v -> sum[k = 1:length(u)] u[k] * v[k]; dot_([1, 2, 3], [4, 5, 6])
32
```

Discussion. A hundred thousand terms of Basel; a double sum with `pick` selecting the diagonal (booleans don't multiply — the bridge is explicit); and a dot product *defined* in sigma notation — the definition reads as the mathematics because it is the mathematics.

Problem 14.4 — The grand combinations. Everything at once:

```
cozy> rng(1); A |> {a = det, b = inv} where A = eig(rand(2)).vectors
{a = -0.998887, b = [-0.621925, 0.784499; 0.812955, 0.584237]}
cozy> ans.a * det(ans.b)
1
cozy> rng(2); sum(-1.96 < z < 1.96) / n where n = 10000, z = randn(1, n)
0.9484
cozy> 1:m ~> (fn k -> prod[j = 1:k] (1 - (j - 1) / 365)) |> (fn p -> find(p < 0.5)[1]) where m = 60
23
```

Discussion. Line one is the thesis of the language in one statement: `rand(2)` makes a matrix, `eig(...).vectors` takes a field of its eigendecomposition, `where` names it `A`, and the fan-out applies `det` and `inv` to the same value — five features composing without a seam, and `det(A) * det(inv(A)) = 1` confirms the algebra on the next line. Line three is the statistician's one-liner: `n` and `z` bound in one `where` clause, the chain `-1.96 < z < 1.96` producing the mask, 94.84% inside. And the last line is the **birthday problem** solved in a single pipeline — map each party size `k` to its all-distinct probability `prod[j = 1:k] (1 - (j - 1)/365)`, then find the first size where it drops below one half: **23**, the famous answer, computed in the notation you'd use to explain it. When the sentence structure of a language matches the thought structure of its user, this is what it looks like.

Problem 12.5 — Composing a function with itself. An operator that returns f f — a function eating a function, producing their composition:

```
cozy> format(4)
cozy> let selfcomp = fn f -> fn x -> f(f(x))
<fn/1>
cozy> selfcomp(fn t -> t + 3)(10)
16
cozy> selfcomp(sqrt)(16)
2.000
cozy> [1, 2, 3] ~> selfcomp(fn t -> t * 10)
[100, 200, 300]
```

Discussion. `selfcomp(f)` *is* a function: apply it, bind it, or send an array through it with `~>`. The n-fold generalization must use `if` rather than `pick` — `pick` is an ordinary function and evaluates *both* arms, which would recurse forever; `if` is the lazy form:

```
cozy> format(4)
cozy> let iterate = fn f, n -> if n <= 0 then (fn x -> x) else (fn x -> f(iterate(f, n - 1)(x))) end
<fn/2>
```

```
cozy> iterate(fn t -> t * 2, 10)(1)
1024
cozy> iterate(fn t -> sqrt(1 + t), 40)(1)
1.618
cozy> (1 + sqrt(5)) / 2
1.618
```

Discussion. Ten doublings make 1024 — and the second line is a small wonder: forty-fold composition of √(1+t) converges to its fixed point, **the golden ratio**, because solves x = √(1+x). `selfcomp` is just `iterate(f, 2)`.

```
cozy> format(4)
cozy> let d = fn f -> fn x -> (f(x + h) - f(x - h)) / (2 * h) where h = 1e-4
<fn/1>
cozy> let selfcomp = fn f -> fn x -> f(f(x));
cozy> selfcomp(d)(sin)(pi / 3)
-0.8660
cozy> -sin(pi / 3)
-0.8660
```

Discussion. Composing the derivative operator of Problem 10.6 with itself yields the second derivative: −sin(/3) on the nose. Note `h = 1e-4` rather than `1e-6` — nesting squares the step, and 10^{-12} denominators drown in cancellation. Operators compose like functions because they *are* functions.

---

15. The Cozy instruments

Everything before this chapter, Cozy inherited: every Neutrino program is a valid Cozy program with the same meaning, enforced by the inherited golden suite on every build. This chapter is what Cozy adds — sparse matrices, dual numbers with exact derivatives, and constrained optimization — the instruments the heavier language was built for. One line of context: `buildinfo().backend` names the linear-algebra backend your binary carries (`tier0` hand-rolled, `openblas`, or `accelerate`); the language is byte-identical under all three.

Problem 15.1 — A sparse system, solved without ever densifying. The 1-D Laplacian on 400 points is tridiagonal: 1198 nonzeros in a matrix of 160,000 cells. Sparsity is legible — the value prints its own ledger — and the conjugate-gradient solver from sparselin.cz touches only `S * v`:

```
cozy> let n = 400;
cozy> let i = [1:n, 1:n-1, 2:n]; let j = [1:n, 2:n, 1:n-1];
cozy> let v = [2 * ones(1, n), -ones(1, n-1), -ones(1, n-1)];
cozy> let L = sparse(i, j, v, n, n)
sparse 400x400, nnz = 1198
  (1,1)  2
  (1,2)  -1
  (2,1)  -1
  (2,2)  2
  (2,3)  -1
  (3,2)  -1
  (3,3)  2
  (3,4)  -1
  (4,3)  -1
  (4,4)  2
  (4,5)  -1
  (5,4)  -1
  ... (1186 more)
cozy> load("packages/sparselin.cz");
cozy> let s = cg(L, ones(n, 1)); s.iters
```

```
200
cozy> s.relres < 1e-9
true
cozy> sl_norm(dense(L) * s.x - ones(n, 1)) < 1e-7
true
```

Discussion. The triplet constructor takes parallel index/value rows; `who` and the echo both speak nnz, never a wall of zeros. CG converges in 200 iterations — n/2, the textbook bound for this spectrum — and the residual check densifies only to *verify*, which is the one honest use of `dense` in a sparse workflow. The promotion law holds throughout: nothing densified silently, and anything that would have (`L + 1`, `L \\ b` direct) is an error naming the explicit route.

Problem 15.2 — Derivatives that are exact, not approximate. A dual number carries a value and a derivative through every operation with `eps^2 = 0`; `d(f)` from autodiff.cz is the one-line consequence:

```
cozy> load("packages/autodiff.cz"); format(6)
cozy> d(fn x -> exp(sin(x^2)))(1)
2.50676
cozy> 2 * cos(1) * exp(sin(1))
2.50676
cozy> let rosen = fn x -> (1 - x[1])^2 + 100 * (x[2] - x[1]^2)^2; grad(rosen)([1.0; 1.0])
[ 0.00000
  0.00000 ]
```

Discussion. The first pair is the point: the machine derivative of $\exp(\sin(x^2))$ and the hand-derived $2x \cdot \cos(x^2) \cdot \exp(\sin(x^2))$ agree to every printed digit because they are the *same computation* — the chain rule executed by arithmetic, no step size anywhere. The Rosenbrock gradient at the known minimum is exactly zero, not small: dual arithmetic is exact, so its zeros are too.

Problem 15.3 — Constrained maximization: a portfolio. Mean-variance selection: minimize risk minus a return bonus, subject to weights that sum to one and stay nonnegative — an equality and an inequality constraint, handled by the augmented Lagrangian in optim.cz:

```
cozy> load("packages/optim.cz"); format(4)
cozy> let mu = [0.10; 0.06; 0.04];
cozy> let Sig = [0.09, 0.01, 0.00; 0.01, 0.04, 0.01; 0.00, 0.01, 0.01];
cozy> let obj = fn w -> sum(w .* (Sig * w)) - 0.5 * sum(mu .* w)
<fn/1>
cozy> let m = minimize_con(obj, [0.3; 0.3; 0.4], {eq = fn w -> [sum(w) - 1], ineq = fn w -> -w}); m.conv
true
cozy> m.x
[  0.2414
  0.08621
   0.6724 ]
cozy> sum(m.x)
1.000
```

Discussion. The `cons` record is the whole constraint language: `eq` driven to zero, `ineq` driven nonpositive (`-w <= 0` *is* `w >= 0`), each an ordinary function returning a column. The optimizer differentiates the augmented objective — kinks and all — with the same dual numbers as 15.2; no Lagrangian algebra was written by anyone. The weights tilt toward the high-return asset exactly as far as its variance allows, and the budget binds to the printed digit.

Problem 15.4 — Estimation: the closure is the estimator. Nonlinear least squares in the pattern that generalizes to GMM and maximum likelihood: a factory takes the *data*, returns the objective, and the optimizer recovers the truth from noise:

```
cozy> load("packages/optim.cz"); format(4)
cozy> let x = (1:20)' * 0.25; let y = 2.5 * exp(-0.8 * x) + 0.01 * randn(20, 1);
```

```
cozy> let make_ssr = fn x, y -> fn b -> sum((y - b[1] * exp(b[2] * x)) .* (y - b[1] * exp(b[2] * x)))
<fn/2>
cozy> minimize(make_ssr(x, y), [1.0; -0.1]).x
[   2.513
  -0.8050 ]
```

Discussion. True parameters (2.5, −0.8); recovered (2.513, −0.805) from twenty noisy points — and the session reproduces exactly, because the RNG is seeded by default. The factory signature *is* the estimator: `make_ssr(x, y)` states what NLLS needs, the returned `fn b` is what optimizers eat, and capture-by-value pins the data without copying it (arrays are refcounted). Swap the inner expression for a moment condition times a weighting matrix and this same shape is GMM; take a log-density and it is maximum likelihood. One idiom, the whole estimation zoo.

Problem 15.5 — Life-cycle consumption: the Euler equation falls out. A five-period consumption-savings problem in the maximal idiom:

```
cozy> load("packages/optim.cz")
cozy> let T = 5; let   = 0.96; let R = 1.04; let W = 100.0
100
cozy> let U = fn c -> sum[t = 1:T]  ^(t - 1) * log(c[t])
<fn/1>
cozy> let sol = maximize_con(U, W / T * ones(T, 1), {eq = fn c -> [(sum[t = 1:T] c[t] / R^(t - 1)) - W]}
cozy> format("fixed", 2); sol.x
[ 21.67
  21.63
  21.60
  21.56
  21.53 ]
cozy> abs(sol.x[2] / sol.x[1] -   * R) < 1e-4
true
cozy> abs((sum[t = 1:T] sol.x[t] / R^(t - 1)) - W) < 1e-6
true
```

Discussion. The whole model is three lines: lifetime utility in sigma notation, the present-value budget as one equality, and the solver. The Euler equation falls out rather than being imposed: consumption growth c_{t+1}/c_t equals R to four digits, and the budget binds to 1e-9. One precedence lesson, learned live: `sum[t = 1:T] c[t]/R^(t-1) - W` subtracts W from EVERY term — the sigma body extends through trailing additive terms, so a reduction used inside a larger expression wants parentheses: `(sum[...] ...) - W`. The first draft of this problem "converged" to a budget five times wealth for exactly that reason.

Problem 15.6 — Minimum variance, numerically and in closed form. The classic allocation problem, solved and then checked against theory:

```
cozy> load("packages/optim.cz")
cozy> let Σ = [0.04, 0.006, 0.0; 0.006, 0.09, 0.012; 0.0, 0.012, 0.16]
[  0.04  0.006      0
  0.006   0.09  0.012
      0  0.012   0.16 ]
cozy> let vol2 = fn w -> sum(w .* (Σ * w))
<fn/1>
cozy> let sol = minimize_con(vol2, ones(3, 1) / 3, {eq = fn w -> [sum(w) - 1]});
cozy> format(4); sol.x
[ 0.6222
  0.2309
  0.1469 ]
cozy> let wstar = (Σ \ ones(3, 1)) / sum(Σ \ ones(3, 1));
cozy> max(abs(sol.x - wstar)) < 1e-4
true
```

```
cozy> format(4); sqrt(vol2(sol.x))
0.1621
```

Discussion. The minimum-variance portfolio, twice: once numerically (the quadratic form in sigma-free matrix idiom, the fully-invested constraint as one bracket), once analytically — w* = Σ ¹1 / 1'Σ ¹1 is a single backslash and a normalization. The two agree to 1e-5, which is the augmented Lagrangian meeting a closed form it never saw. The low-volatility asset takes 62% of the book; the noisiest, 15%.

Problem 15.7 — GMM from generated data: two steps and a J test. Simulate an endogenous regressor, then estimate by generalized method of moments:

```
cozy> load("packages/optim.cz"); rng(7)
cozy> let n = 400; let z1 = randn(n, 1); let z2 = randn(n, 1); let Z = [z1, z2]
[   -0.151573    0.0447116
     0.829897    -0.663658
       0.5871   0.00904516
   -0.0700575    -0.906172
    0.0944719    -0.551759
   -0.0971522    -0.309004
       1.8753     -1.04261
      1.44401    -0.941383
     0.779201     0.348058
      1.09863    -0.916612
    -0.126008    -0.340744
     -1.10065      1.09935
     0.260005   -0.0403893
    -0.241558      3.00297
     -1.17008     0.244701
    -0.470762     0.704433
     -1.61223    -0.215406
     0.347018     0.198145
      1.28542    -0.057098
      1.43426     0.732928
     -1.09092     0.781457
     -1.52875      1.64148
    -0.174465    -0.684616
     0.882119    -0.160695
     0.524001    -0.942201
     0.549389    -0.729504
     -2.13068     0.580856
     -1.36111      -0.1985
     -1.85043    0.0163428
     0.236856     0.610833
      1.43762     -1.87085
      1.67445     -2.82639
     0.620312    -0.876323
   -0.0609152     -1.13611
    -0.367133     0.415352
     0.187581   -0.0823624
    -0.545965     0.870825
     -1.33298       -1.261
     0.405574       1.7892
   -0.0427913     0.860082
     0.104665     -1.92831
     0.478408     -1.16145
     0.478359       1.7187
```

```
  0.966273     -1.14534
   0.86049    0.0183171
  0.271869    -0.861087
  0.330344    -0.235065
 -0.694284      1.28232
 -0.287464     0.168501
 -0.657692    -0.604422
  0.522236     0.567704
 -0.819537    -0.353122
   1.59002     0.434134
 -0.322473     0.402142
  0.434898     0.370435
 -0.885685     -1.16283
  -2.13329    0.0569353
 -0.181141    -0.468656
  0.581651      1.29733
  -0.32149     -1.57091
  -1.86805    -0.442633
  -1.05315    0.0581589
  0.109609     0.883949
 -0.362048     0.367516
   1.38674    -0.323672
 -0.353629    -0.622189
0.0412343    -0.654359
  0.360656      -1.1121
 -0.513953     -2.20494
   -1.2684     -0.13809
  0.595906     0.561962
  -1.93633     0.706905
  -1.03647      1.16709
   1.25551  -0.00984879
 -0.554666     -1.36779
 -0.956376     -1.55836
  -0.60137     -2.30183
 -0.453789     -0.76224
-0.0502956      0.27254
   1.54731    -0.494882
   2.45658     0.676914
  0.721731     -0.37598
  -1.15504   -0.0792247
 -0.452863      -1.5092
  -0.54885    -0.189737
  0.522068     0.344054
  0.959158    -0.186883
   0.65558    -0.146098
   1.14179      1.03187
   1.12632      1.35815
  0.395319     -1.44389
  0.528145       -0.704
  -1.64131     -1.04023
 -0.917711    -0.540956
   1.23235     -1.43204
  -0.50797     0.811155
   1.52559     0.412204
  -1.77288   -0.0613725
```

```
-0.595189    -0.938482
-0.419747       1.3687
 0.750481    -0.753265
-0.364997      1.36517
 0.275119     0.162555
 0.412824     0.968603
 0.381629     0.266923
-0.922986     0.743231
 0.965927    -0.039943
-0.280916     -1.42109
  1.30978     0.643831
-0.710212    -0.368089
  2.14988    -0.241976
 0.261303    -0.239688
  1.18126     -1.41982
-0.715686    -0.548738
  -1.1878     -1.57277
0.0992219    0.0613226
 -1.10256    -0.170133
 0.705548      2.35778
 0.838803     0.390286
 0.392377      2.20092
-0.522877      1.66756
-0.900918     -1.04563
 0.735325     0.163481
  1.25066      1.36175
  1.72932      1.82576
 -1.26506     0.942944
 -1.56242      1.53924
 0.643119    -0.140833
  1.05781    -0.331514
0.0173866     0.586124
 0.970864     0.318459
-0.222209     0.236432
  2.46985    -0.343488
 0.667513      1.22019
-0.254733     0.717178
  1.24094    -0.502937
 0.270884    -0.282947
 0.337867     -1.26267
  1.51573     -1.01021
-0.057455    -0.488204
 0.778103     0.785291
 -0.26375   -0.0509522
-0.287289      2.29243
  1.83475     0.554635
-0.232136     -1.15541
 -1.26909     0.619056
  1.33993     0.851311
 -0.90212      1.24739
-0.467963     0.233333
 0.360561     -1.35722
-0.825019    -0.339522
 0.682017    -0.139573
-0.426945    -0.139924
```

```
-0.102219        0.316
-0.749759     0.184778
-0.761958     0.820747
-0.873397    -0.568729
-0.167778     -1.28175
 0.200775     0.881143
-0.982687   -0.0249252
   -1.708     0.350411
 -1.60591     -1.49946
-0.355011      0.91133
 -0.18876     -1.85489
 0.393301     0.442085
  1.17651     0.703291
  1.58322   0.00570613
-0.687907    -0.994323
 -2.01484      1.49292
0.0870042    -0.495478
 -1.40335      -1.0777
0.0164339     0.356458
 -1.03619    -0.968692
-0.942275     -2.27883
 -1.06647     -0.21991
  2.58831    -0.900531
-0.245872      1.72749
 0.685798    -0.122529
 -1.35895     0.638468
  -0.6709     0.028702
 0.750344     0.975299
 0.260838    0.0660235
 0.390328     -1.71004
 -1.58208     -1.87128
 0.978849     0.793077
  0.28229       1.1777
0.0616104     0.962765
 0.278376    -0.937609
-0.442256      1.73469
  1.14728   -0.0249743
 0.857786    -0.556192
-0.701854     0.428491
  1.19735     0.774148
-0.387888  -0.00347676
-0.756597     -2.86869
-0.619207      1.25812
 0.690129    -0.514691
 0.598463     0.790499
-0.559837     -1.10234
-0.706069      -0.2852
  -1.2701     0.426677
 0.864395      1.46651
 0.260101    -0.420732
  2.08249     0.581723
   0.5633    -0.864267
 0.344934    -0.157891
 0.601087     -1.49198
 0.467999    0.0694325
```

```
  0.317653    -0.330911
   2.25271    -0.803305
   2.13548     -1.16472
   0.37795     -1.15568
 0.0592657      2.32216
   1.51248      1.46903
  -1.63947    -0.895687
 -0.289042    -0.927242
   1.62675    -0.502993
   0.76005   -0.0488547
   1.18676    -0.556665
   1.12228    -0.960943
   1.11366      1.66303
   -2.1484      1.61746
  0.572218     -2.03397
 -0.964805    -0.420126
  0.207691     0.305803
  0.385258    -0.124085
  0.717677   -0.0451656
  0.954863    -0.396568
  -1.47768     0.614237
  -2.70992    -0.726763
  0.345497    0.0334373
-0.0198204    -0.192064
  -2.24441      1.33182
  0.514827     -0.76665
   0.12367      -2.3107
 -0.407179    -0.882485
   -2.3629     0.229338
 -0.531538     0.147666
  -0.26192     0.473617
 -0.165158      -1.2275
  0.120397     0.490137
  0.290628    -0.653065
  0.769227     0.877308
 0.0385978     -1.21864
 -0.800794      1.12154
 -0.139936    -0.628172
   1.55519      1.26595
  0.431658    -0.739076
  -1.09354    0.0971188
  0.900439     -1.03822
  0.760742    -0.674976
 -0.688059     -1.14356
  0.625163      1.30704
  0.568291     0.453967
 -0.953458     0.815865
  0.179744    -0.454425
   0.80113      1.18885
 -0.316603     0.748825
 -0.820036     0.359578
   0.42709   -0.0494656
 -0.273447    -0.423184
   1.74418    -0.415855
 -0.800071    0.0105151
```

0.444528	0.254712
1.67289	-1.09971
1.16237	-0.701065
0.777351	0.592909
0.732448	0.975445
-0.00480243	0.573617
-0.673645	-1.36879
-0.342362	-0.294719
0.195007	1.19869
-1.21709	-0.440525
2.95609	-0.0902644
0.861216	-0.735773
0.7971	-0.774015
-1.14338	-0.432496
1.69509	-1.15728
0.250746	-1.44679
0.0821588	0.105657
1.00552	1.71133
-0.631494	2.30164
0.508804	0.0837307
0.302711	0.909592
0.0700167	0.61009
-1.80064	-0.321736
0.41629	0.417565
0.650227	0.959119
-0.618663	0.236219
-1.11328	-0.284214
-0.466767	-1.14361
0.584981	-0.168078
0.489013	-1.83603
-1.76792	1.65968
0.649628	1.55515
0.421061	1.81912
-1.89675	-0.643441
0.124758	0.934472
0.63096	-1.06039
-0.58923	-1.27537
-0.3913	1.70391
0.872412	-1.12208
0.87092	0.765092
0.150133	0.662558
-0.201332	-0.402004
-0.777776	0.0557404
-0.631318	1.90258
-1.21801	0.947881
-0.568658	-0.465009
-0.325056	-0.990836
-0.819698	0.294029
-1.28824	-1.74996
-0.412871	0.711924
-0.681366	-0.953975
-0.628542	1.53709
-1.08332	-0.426776
-0.307464	1.24207
-0.435811	-0.127368

0.0405573	-2.3251
-0.153146	-0.550013
-0.923234	0.108574
-1.14844	0.987642
1.1065	-0.976228
0.955822	0.571704
-1.16665	-0.726496
0.350085	1.11629
-1.88258	-0.128461
-1.27695	0.242985
-0.125031	0.847292
-1.96151	-0.633188
-0.866549	-0.881657
1.27625	0.430836
-1.22767	-1.12834
-1.10397	-0.816363
1.57499	-0.494328
0.87063	0.108788
-0.0152481	0.412263
-0.418132	-0.411219
-0.80097	-0.212385
-0.207877	-2.10247
-1.53716	-0.668418
-0.630422	0.90268
1.6597	2.50557
0.48481	-0.204677
0.660956	-0.111887
0.500982	1.96739
1.5485	-0.804572
0.768103	-0.962477
-1.21487	-0.181101
-0.314929	-1.66787
-0.395228	-0.460087
-0.218627	-0.861413
-0.490703	-0.0164957
-0.168772	1.21234
1.03841	0.170454
-0.027441	-1.06639
2.77899	0.200726
-1.40124	0.55948
-0.4679	1.39873
-0.917867	0.805248
-0.809265	-0.403207
0.336962	-0.627153
-0.728145	-0.657834
0.132569	-0.00335447
0.318755	-2.38912
2.06966	0.072687
-0.716724	-0.964165
0.0307231	-1.2281
-1.42953	-0.390029
0.636762	-1.31316
1.17148	0.328183
-0.429528	-0.812855
-0.596175	1.26193

```
      1.73562    -0.885951
      2.20544    -0.600008
     0.982703    -0.619253
     0.603715   -0.0373096
     0.589184     -1.95754
     -1.60122     0.655383
   -0.0173868     0.579563
     0.293006       1.7943
    -0.824108     -1.42178
     -0.12921    -0.965722
     -1.62922     -1.66257
     0.142683     0.250599
    -0.309601      1.55108
    -0.454722     -2.12191
     -1.37801      0.21924
     0.559711    -0.398719
     0.500906     -1.60235
    -0.129558      0.56783
    -0.567194      1.90569
     -2.04023   -0.0941446
      -1.1553      -1.1521
    -0.569464     -1.52068
     -0.59943      1.02151
    -0.160384      -1.0846
     -3.05962    0.0455267
    -0.642902     0.931932
     0.646721    0.0827786 ]
cozy> let v = randn(n, 1); let x = z1 + 0.5 * z2 + v; let y = 1.5 * x + 0.7 * v + 0.3 * randn(n, 1)
[    1.72867
      4.5705
    -1.55555
     1.54526
     1.90313
     3.25234
     3.80681
    -2.43119
     1.84653
   -0.658165
    0.149619
    -1.94998
  -0.0909338
     4.49357
    -4.71578
    0.548396
    -2.87483
     5.05618
     5.61986
     3.59533
     1.70737
   -0.035204
    -1.21194
    -1.22696
     2.16899
   -0.734868
    -5.38028
```

-3.07371
-3.82576
-4.39838
0.520364
-0.414644
-5.3663
-1.87731
3.04423
0.518483
-2.98746
-6.09108
-0.270015
-1.37767
1.10634
-1.9322
8.03846
-0.0394637
6.98003
0.350434
1.17257
1.60016
-1.7279
-5.98211
4.43996
-2.16029
3.98318
-1.46993
0.295552
-2.49484
-4.00535
-3.55676
-1.34453
-3.48686
-3.73704
-2.81034
0.327264
-0.324423
1.31182
0.128558
1.50462
3.97618
2.86673
-3.74811
-1.25947
-0.136051
-3.25292
-1.54655
-3.1214
-3.49837
-0.749712
-0.689413
-2.0263
-4.13715
5.09233
1.48002

-1.97285
-4.94385
0.368798
0.736708
2.77567
1.30755
5.18147
2.58477
-4.42465
1.78534
-3.271
0.92828
-0.496413
0.15238
3.87901
1.49756
-2.90604
1.71503
-1.30672
-1.47105
-2.21289
0.0607325
3.02665
0.500692
-3.72941
1.95896
2.81021
-4.065
2.7043
-0.991475
-4.213
-2.67042
-5.22261
-0.706666
-4.3829
-0.285904
-0.280305
6.25447
3.44563
-3.45368
0.698363
4.03908
6.14039
-2.6221
-0.364207
2.88831
1.22607
1.66179
2.1067
-0.584588
7.25885
1.70427
0.893328
1.80692
0.414431

-4.40421
4.03224
-3.14599
-2.82983
0.493235
-0.882802
0.22081
-2.15499
-2.68502
2.9104
-2.25607
0.778308
-0.941922
2.49996
-2.23279
-2.10837
2.29364
-0.513646
-3.8971
1.46852
-4.11729
-1.26609
0.0546292
-0.193194
-1.19716
1.99764
-3.487
0.305299
1.61698
2.86442
-4.72308
-1.47193
-0.875103
0.379329
1.72539
1.1672
-4.21359
-4.22074
2.42286
2.16962
2.00241
-0.689028
-5.84883
2.48568
0.989111
0.191524
-3.49785
4.02867
0.211752
0.816829
-0.388607
-2.94866
2.86349
0.0292994
-0.028726

```
  4.91819
   5.1629
  -5.6019
  1.89398
  3.82441
  3.51431
-0.705719
 -3.19256
 -1.24741
 -1.36923
 -2.16331
  8.96499
-0.519514
  3.76687
 -2.04045
 -2.36989
 -3.31767
  1.51674
  6.09656
 -1.04571
  3.10568
  5.42663
 -2.53796
 -2.79879
 0.620132
  1.73807
 -1.10201
  2.76965
  4.44254
 -3.63424
-0.253909
 -1.49783
  2.43432
   1.4061
  0.07221
 -2.00575
 0.115759
 -6.07894
 -1.18395
  -1.2351
 -1.73768
  1.06777
 -2.24504
 -4.97459
 -8.13518
  2.17409
  1.91663
  1.95644
  2.76584
  0.25582
  2.94507
-0.585624
  -3.9427
-0.974048
  1.36702
```

-1.14827
-3.73352
2.0861
0.442896
-0.44933
-1.29198
4.31414
1.50405
-1.10297
0.103301
-1.03129
1.83871
-3.09595
-0.76174
1.61944
-0.739826
-0.470317
-0.556953
-3.24488
-1.17239
1.16499
1.86201
-3.5572
-2.11994
-0.69154
-5.16985
1.76473
2.77052
-0.410289
-0.495444
1.82902
-2.34154
0.308152
1.14379
2.07731
1.88189
3.25487
2.62198
-5.81622
-3.19984
1.58519
0.0606411
1.66578
0.937508
-1.17845
-1.46518
-0.590466
4.01811
0.317456
-5.18267
-0.323524
-3.78225
1.02591
2.88528
-2.75539

```
  2.13056
 -1.66214
 -1.12551
 -2.56195
  3.98048
 -1.15365
 -2.03173
0.0168187
  1.43177
  -3.0139
 -3.49865
  -2.5668
-0.310277
 -2.69562
  1.54154
-0.334342
 -3.87624
 0.250165
  3.06039
 0.220319
  1.79891
  6.20509
-0.144198
 -1.67434
-0.853579
 -4.44195
 -1.90911
 -3.29791
 -1.79922
  1.88566
 -1.32965
 -4.70986
  5.14362
  2.20576
0.0965242
-0.293755
-0.886212
 -1.57989
 0.302391
 -1.84791
  3.88765
 -1.50977
 0.465343
    3.0328
  6.51725
 0.700939
 -4.38458
  2.76607
 0.400117
  1.19835
  1.54807
 0.179585
-0.0442415
 -2.02767
  5.05219
```

```
  -0.0613324
    -2.10511
    -3.97148
    -1.09154
     3.98957
    -5.03046
    0.942265
     3.63793
     5.41345
    -2.22587
    -2.48153
    -2.31388
     1.42576
    -4.08586
    -4.48139
     2.86857
   -0.465998
    -1.42177
    0.959853
    -3.45883
     2.43721
    -2.93798
    -3.81933
    -2.41463
    -2.35162
    0.924688
    -4.42656
      1.9566
   -0.349898
    -4.99545
   -0.216075
    0.772268
    -3.68041
    0.271639
     4.55381
   0.0310894
    -5.76132
    -2.42847
    0.722207
     -1.5307
    -4.58729
    -4.05459
   -0.686802 ]
cozy> let g = fn b -> Z' * (y - x * b) / n
<fn/1>
cozy> let  1 = minimize(fn b -> sum(g(b[1]) .* g(b[1])), [0.0]).x[1];  1
1.50162
cozy> let e = y - x *  1; let M = Z .* (e * ones(1, 2)); let Si = inv(M' * M / n);
cozy> let fit = minimize(fn b -> sum(g(b[1]) .* (Si * g(b[1]))), [ 1]);
cozy> fit.x[1]
1.50256
cozy> n * fit.fx
1.12386
```

Discussion. Generated data with a genuinely endogenous regressor (the shared shock v), two instruments,

one parameter — overidentified GMM in the maximal idiom: the moment function is one line of matrix algebra, the first step minimizes g'g, and the second step reweights by the inverse moment covariance. Note the design lesson the gate taught: the dual-number gradient cannot pass through \, so the weight matrix is inverted ONCE outside the objective — precompute what does not depend on the parameter, and let autodiff flow through matmul. Both steps land within a standard error of the true 1.5, converging in two iterations each, and n · Q 1.12 is the Hansen J statistic on one degree of freedom: the overidentifying restriction is comfortably unrejected, as it should be, since the model is true.

Problem 15.8 — Taylor by quotation: the language reads its own sin. Quotation, symbolic differentiation, and eval close a loop:

```
cozy> load("packages/symb.cz")
cozy> let f = ast(fn x -> sin(x)).body
{op = "sin", l = {op = "var", name = "x"}}
cozy> let dtree = fn e, k -> if k == 0 then e else dtree(ddx(e), k - 1) end
<fn/2>
cozy> let x = 0.0;
cozy> let c = (0:7) ~> (fn k -> eval(show(simp(dtree(f, k)))) / gamma(k + 1)); c
[0, 1, 0, -0.166667, 0, 0.00833333, 0, -0.000198413]
cozy> let Tn = fn m, u -> sum[k = 0:m] c[k + 1] * u^k
<fn/2>
cozy> abs(Tn(7, 1.0) - sin(1.0)) < 3e-5
true
cozy> let xs = -pi:0.1:pi;
cozy> plot(xs, [xs ~> sin; xs ~> (fn u -> Tn(1, u)); xs ~> (fn u -> Tn(3, u)); xs ~> (fn u -> Tn(7, u))]
  sin and its Taylor truncations
     3.06 |                                                                  ++
     2.69 |                                                               ++++
     2.33 |                                                            ++++
     1.96 |x                                                         +++
      1.6 | x                                                     ++++
     1.23 |  x                                                ++++
     0.87 |   xx                                          +#################
    0.505 |     x                                     #####            xxx ####
    0.141 |#     xx                                ####                   xx   ###
   -0.224 | ####   xx                         ###                          x
   -0.589 |     #### xxx                #####                             xx
   -0.953 |         ###############+                                        x
    -1.32 |                 ++++                                            x
    -1.68 |             ++++                                                 x
    -2.05 |          +++
    -2.41 |      ++++
    -2.78 |  ++++
    -3.14 |++
          +-----------------------------------------------------------------
           -3.142                                                      3.058
  * series 1 (sin)
  + series 2 (T1)
  x series 3 (T3)
  # series 4 (T7)
```

Discussion. The analytical engine end to end: `ast` quotes the typed sin into a tree, `ddx` differentiates the TREE k times, `show` prints the derivative back to source, and `eval` runs that source at x = 0 — so the Taylor coefficients are computed by the language inspecting its own functions, and they come out EXACT: 1, -1/6, 1/120, -1/5040. The truncations are then one sigma each, and the plot shows the classic picture: T1 is the tangent line, T3 peels away past |x| = 2, and T7 is indistinguishable from sin across the whole period until

the edges. Seven digits of sin(1) from an eighth-degree tail bound, and not one derivative was approximated.

Problem 15.9 — Probit by Newton: exact scores, exact information. Maximum likelihood with every derivative exact:

```
cozy> load("packages/optim.cz"); load("packages/dist.cz"); rng(12)
cozy> let n = 600; let  true = [0.5; -1.0]
[ 0.5
   -1 ]
cozy> let X = [ones(n, 1), randn(n, 1)]
[           1      0.57324
            1     -1.42267
            1     0.433214
            1    -0.186217
            1    -0.152374
            1      1.57513
            1      0.34454
            1    -0.697393
            1    -0.414509
            1     -1.00098
            1     0.225492
            1     0.295028
            1     0.665954
            1     0.428043
            1     -0.35908
            1     0.941001
            1     0.400239
            1      0.63685
            1    -0.861972
            1      1.86238
            1    -0.416505
            1    -0.965693
            1      1.19597
            1     0.198229
            1    -0.939412
            1     -1.97707
            1     0.597734
            1    -0.653571
            1      0.58235
            1     0.436539
            1     0.458517
            1    -0.312622
            1      1.23157
            1    -0.217799
            1    -0.225964
            1      3.51283
            1    -0.569889
            1     0.211478
            1      1.54476
            1     -1.43354
            1   -0.0192153
            1    -0.272201
            1    -0.835284
            1     0.818589
            1     0.832003
            1      2.42976
```

```
1    -0.239401
1     0.672871
1    -0.570648
1      1.65268
1    -0.181917
1     0.521961
1      1.80093
1     0.209185
1    -0.840696
1     -1.56333
1     -1.29017
1    -0.515871
1    -0.165986
1      -1.4825
1     0.472184
1     -1.46842
1     -1.99013
1     0.451565
1     -0.39717
1      1.12718
1    -0.747539
1     0.093148
1   -0.0725952
1      1.07753
1     -1.07981
1     0.223233
1    -0.902975
1    -0.160884
1      1.13838
1    -0.192741
1    0.0503237
1    -0.219283
1     0.281825
1     0.601588
1     -1.49205
1       1.1937
1     0.757233
1     0.391893
1     0.733395
1    -0.934636
1     0.104257
1    -0.576543
1    -0.294866
1      0.43811
1     -1.60578
1    0.0742366
1      1.03839
1    -0.334847
1      0.64101
1    -0.461002
1      1.54824
1     0.504848
1    -0.888167
1     -2.07486
1      2.60661
```

```
1     0.356215
1     0.440748
1    -0.722507
1      1.05841
1     0.588498
1      1.30116
1     0.328273
1     0.264689
1    0.0712892
1     0.928206
1      2.21444
1     -1.04656
1     0.578774
1    0.0169521
1    -0.218614
1     -0.80593
1       0.7616
1    -0.665303
1     -0.54399
1     0.595517
1     0.181784
1    -0.586261
1    -0.191746
1     0.415716
1      1.00454
1     -1.19511
1      1.12642
1     -0.59242
1     -1.30257
1    0.0237476
1     -1.29272
1     -1.45752
1       0.8191
1     0.543183
1    -0.264825
1     0.121944
1    0.0311585
1     0.483739
1    -0.125563
1     0.712165
1    -0.357525
1     0.953839
1     0.375214
1     0.826485
1     -1.85046
1    0.0174853
1     0.486786
1      0.13323
1      2.30605
1     -0.54833
1     -0.94673
1     0.637348
1     -2.18547
1      2.43017
1      1.55239
```

```
1     0.193277
1    -0.381683
1     0.669287
1     0.459287
1     -2.98076
1    -0.230773
1     0.746881
1     0.153409
1    -0.431403
1     0.763501
1     -1.22404
1     -0.38364
1       -1.554
1     0.510194
1     0.109054
1    0.0152903
1      0.35086
1      2.02558
1    -0.549223
1     -1.19026
1      1.24984
1      -1.7595
1    -0.201041
1     0.519209
1      -1.4974
1     0.034774
1    -0.469744
1     0.845738
1      2.31345
1     -1.38172
1     0.137831
1   -0.0575792
1    -0.338597
1    -0.346589
1      1.19256
1     -1.16781
1     -1.63701
1    -0.407912
1      -1.2574
1     0.011573
1    -0.596258
1      1.69158
1     0.993333
1     -1.44707
1     0.239412
1      1.85946
1    -0.619069
1     0.398244
1    -0.587858
1      1.50552
1     -1.25242
1      1.06576
1     0.387972
1    -0.880492
1     0.206171
```

```
1     -2.12553
1      1.25424
1     -1.36796
1      1.17325
1   0.00478592
1    -0.409277
1    -0.220047
1     -1.55962
1    0.0556614
1    -0.953617
1    -0.275508
1     0.311049
1      1.24831
1    -0.106425
1    -0.255554
1    -0.270645
1     -2.93118
1     -1.10661
1     0.285826
1     -1.70657
1    -0.803043
1    -0.793969
1    -0.429512
1      0.28372
1    -0.251822
1     0.232097
1    -0.915408
1      1.79392
1     0.784334
1      0.56868
1      0.27006
1    -0.359003
1     0.383156
1     0.377834
1   -0.0507121
1    -0.415455
1     0.416273
1     0.638307
1      1.07262
1     0.877842
1    -0.885696
1    -0.222409
1     0.544131
1     0.411807
1   -0.0134148
1    -0.257945
1    0.0251331
1    -0.381711
1      1.00913
1     0.211603
1     -1.62749
1     0.148533
1     0.709587
1     -1.03537
1      1.72716
```

```
1    -0.319954
1     -1.15003
1    -0.365789
1     0.480208
1    -0.600061
1      1.21238
1     -1.09915
1     -1.08709
1     -0.25185
1    -0.387713
1    -0.857913
1     0.070133
1     0.698272
1      2.14469
1    -0.325931
1    -0.409554
1    -0.858945
1     0.915837
1    -0.879839
1     -1.68537
1    -0.410715
1     0.861024
1      1.56005
1     0.203785
1    -0.532026
1      1.25424
1     0.123129
1   -0.0895196
1     0.442904
1    -0.989014
1     0.140226
1     0.271929
1     0.335956
1    0.0899676
1      1.56757
1      1.30155
1      1.08597
1    -0.748484
1    -0.736824
1     0.796114
1      1.59275
1    0.0881832
1      1.23314
1    -0.239336
1    -0.392717
1    -0.202854
1     -0.53787
1     0.322516
1    -0.634293
1    -0.441796
1    -0.734003
1     0.285121
1      2.14746
1    -0.107587
1     -0.53586
```

```
1    -0.653066
1    -0.927866
1    0.0678099
1    -0.234684
1      1.98983
1     0.682477
1    0.0970999
1     -0.97076
1     -1.33061
1     0.421643
1      0.18318
1      1.45375
1     0.504118
1    0.0409194
1     0.188625
1    -0.584833
1    -0.725263
1      1.43842
1     -1.73895
1    0.0429341
1     0.480126
1      1.45546
1    -0.643586
1      1.36971
1     0.372517
1    -0.854239
1      -1.8631
1     0.259356
1     0.192025
1    -0.623825
1      1.33876
1     0.619938
1    -0.385855
1    -0.638261
1    -0.201278
1     0.379068
1     0.376736
1      0.35088
1    0.0688432
1     0.100065
1    -0.325085
1     0.651085
1     0.520765
1      1.69804
1      1.59488
1      0.69536
1     0.484592
1      1.55203
1     -1.36388
1     0.158528
1    -0.935871
1      1.07947
1      1.56162
1     0.529965
1     0.201195
```

```
1      1.23363
1      0.88411
1    -0.888608
1    -0.784901
1    -0.277704
1      1.13731
1    -0.748156
1     0.764598
1    -0.530091
1    -0.288775
1    -0.720034
1     -1.71723
1    0.0668972
1      1.46734
1      0.19493
1    0.0939873
1     0.557212
1      1.13183
1     0.115264
1    -0.516424
1     -2.14783
1     -1.25389
1     -1.14576
1    -0.751181
1     0.132842
1     0.577106
1     -1.55518
1     -2.31753
1      1.90606
1     0.573725
1     -1.56126
1     -1.26049
1    -0.861748
1     0.108079
1    -0.319282
1      1.04008
1     0.690007
1     0.527153
1      1.31986
1      1.27122
1     -1.11873
1    -0.675805
1    -0.178776
1  -0.00859339
1     0.665887
1    -0.378736
1     0.256387
1    -0.238678
1      1.42554
1    -0.497941
1     0.220293
1     0.371375
1     0.201851
1    -0.286832
1    -0.444971
```

```
1       1.4789
1     0.322292
1     0.699033
1     -1.30775
1     -2.03976
1     0.677504
1    -0.376812
1    0.0390553
1     0.728421
1      0.46172
1    -0.315251
1      0.21321
1     0.682055
1    -0.600504
1    -0.250458
1     -1.13017
1      1.28657
1      1.35246
1    -0.238733
1    -0.746096
1     0.348407
1    0.0275631
1      2.25195
1     -0.40085
1       1.0432
1    -0.521268
1     -1.15571
1    0.0224036
1    -0.186811
1     0.409714
1      1.82015
1     0.512383
1    -0.196767
1      -1.8327
1    -0.904267
1      0.31577
1    -0.454627
1    -0.489881
1      1.56976
1    -0.805916
1     0.470995
1    -0.538278
1      2.22844
1      1.87162
1     -2.40445
1     0.488086
1    -0.565024
1      1.37543
1   -0.0233683
1     0.652703
1     -2.61277
1     -0.57401
1    -0.533135
1   -0.0270004
1      1.22018
```

```
1    -0.479126
1     0.801273
1     0.714668
1    -0.329196
1     0.360631
1    -0.993589
1     -1.39718
1    -0.751761
1    0.0456553
1     -1.16463
1     -1.10483
1    -0.959381
1    -0.826047
1      1.02541
1    -0.475747
1      1.44217
1     0.179435
1      1.26952
1     0.217671
1      1.82732
1      1.16006
1     0.209736
1      1.95314
1     0.739285
1    -0.415974
1      1.11079
1    -0.432276
1     0.538912
1      -1.0209
1     -1.71726
1     0.439237
1    -0.302961
1     -1.33981
1    -0.490467
1    -0.391457
1     0.346442
1      1.95813
1    -0.992226
1     0.931933
1      -1.1898
1    -0.136466
1     0.336862
1     0.285265
1      0.27283
1     -1.95499
1      2.90405
1   -0.0482194
1    -0.283442
1     -1.15193
1     0.345006
1     -1.33619
1      2.11077
1       1.1177
1     -1.22414
1     0.172457
```

```
1      1.21575
1     0.865889
1      1.09496
1   0.00938289
1      1.46349
1     -1.00038
1    -0.217036
1      1.91253
1    -0.596188
1     0.169136
1     -1.77006
1     0.858156
1     0.875893
1       1.0459
1     0.184313
1     0.648475
1    0.0173257
1    -0.589697
1     0.481808
1   -0.0856601
1     0.530596
1     -0.77202
1     0.236386
1    -0.476358
1    -0.674693
1     0.675254
1      1.07975
1    -0.766886
1     0.724551
1     -2.33738
1     -1.12725
1   -0.0569741
1     0.988884
1      1.74219
1     0.431196
1      1.22514
1     -2.01936
1     0.263683
1    -0.595626
1      0.86403
1    -0.328591
1       2.3336
1     0.583792
1      0.02121
1       2.0474
1     -2.01881
1     0.446162
1    -0.565357
1     -1.07789
1    0.0865007
1    -0.758237
1   -0.0208052
1     0.385311
1    -0.531827
1   0.00391172
```

```
            1     0.032458
            1    -0.837241
            1     0.297808
            1      1.61634 ]
cozy> let   = randn(n, 1); let y = pick(X *  true +   > 0, 1.0, 0.0);
cozy> let Φ = dist_Phi
<fn/1>
cozy> let   = fn b -> sum[i = 1:n] (let p = Φ((X[i, :] * b)[1, 1]); y[i] * log(p) + (1 - y[i]) * log(1 -
<fn/1>
cozy> let fit = minimize_newton(fn b -> - (b), zeros(2, 1));
cozy> fit.x
[  0.421748
  -0.974894 ]
cozy> fit.iters
5
cozy> let V = inv(hess(fn b -> - (b))(fit.x));
cozy> sqrt([V[1, 1]; V[2, 2]])
[ 0.0613521
  0.0809202 ]
```

Discussion. The instrument the Hessian machinery was built for. The log-likelihood is written exactly as the textbook writes it — sigma over observations, Φ from the distributions package under its Greek alias — and then everything downstream is exact: `minimize_newton` drives it with true second-order steps (five iterations for 600 observations), and the standard errors come from the observed information matrix, `inv(hess(- ))` at the estimate, with the Hessian computed by hyper-dual arithmetic rather than finite differences. Both coefficients sit within 1.3 standard errors of the truth. No gradient was differenced, no Hessian approximated, and the entire estimator — likelihood, optimizer, covariance — fits on one screen.

Problem 15.10 — Least absolute deviations, without a derivative. The L1 location objective has a kink at every data point — exactly where gradient methods stumble and derivative-free methods shine. BOBYQA, from the NLopt backend, needs only function values and bounds:

```
cozy> rng(4); let d = randn(1, 500) * 2 + 3;
cozy> let lad = fn x -> sum(abs(d - x[1]))
<fn/1>
cozy> let fit = nlmin(lad, [0.0], {alg = "bobyqa", lb = [-10.0], ub = [10.0]});
cozy> format(5); fit.x[1]
2.9216
cozy> abs(fit.x[1] - median(d)) < 1e-3
false
cozy> fit.converged
true
```

Discussion. The minimizer of Σ|d − x| is the sample median — a classical identity, confirmed here to 1e-3 by an optimizer that never saw a derivative. The fence is tagged `cozy-nlopt`: it verifies on builds carrying the backend and is skipped (announced, never silent) elsewhere, and the demo leaves it book-only — the first build-conditional transcript, a mechanism entry 14's residue demanded.

Problem 15.11 — A quarterly series, with the holes left in. Real data arrives dated and gappy. `data/gdpq.csv` is a small quarterly GDP table with one missing cell; `readtable` types the ISO date column as dates and reads the hole as missing, and everything downstream has an honest answer for it:

```
cozy> load("ts")
cozy> let t = readtable("data/gdpq.csv")
{date = [2022-01-01; 2022-04-01; 2022-07-01; 2022-10-01; 2023-01-01; 2023-04-01; 2023-07-01; 2023-10-01;
cozy> quarter(t.date[3]); weekday(t.date[1])
6
cozy> let g = growth(t.gdp)
```

```
[nan; 1.2; 0.889328; nan; nan; 1.05871; 0.761905; 1.0397; 1.21609; 0.831793; 1.19157; 1.26812]
cozy> g[2:4, 1]'
[1.2, 0.889328, nan]
cozy> format(4); mean(g, "omit")
1.051
cozy> let full = dropmissing(t); length(full.gdp)
11
cozy> let h = hpfilter(full.gdp, 1600); format(4); std(h.cycle)
0.2014
cozy> abs(t.date[5] - t.date[1] - 365) <= 1
true
```

Discussion. The growth column is NaN exactly where it must be — at the head (no previous quarter) and across the hole, which poisons two growth rates (its own and the next quarter's base). `"omit"` averages what is real; `dropmissing` hands the HP filter a clean column. Dates carry their own algebra: the closing check is `date - date` arithmetic, and `plot(t.date, t.gdp)` labels the axis with calendar dates on every backend. Entries 15 and 16 of the docket, doing one workday's job together (the ts package: `lag`, `diff`, `growth`, `movavg`, `hpfilter`, `acf`, `pacf`, `arfit`).

Appendix A. Finance (finance.cz)

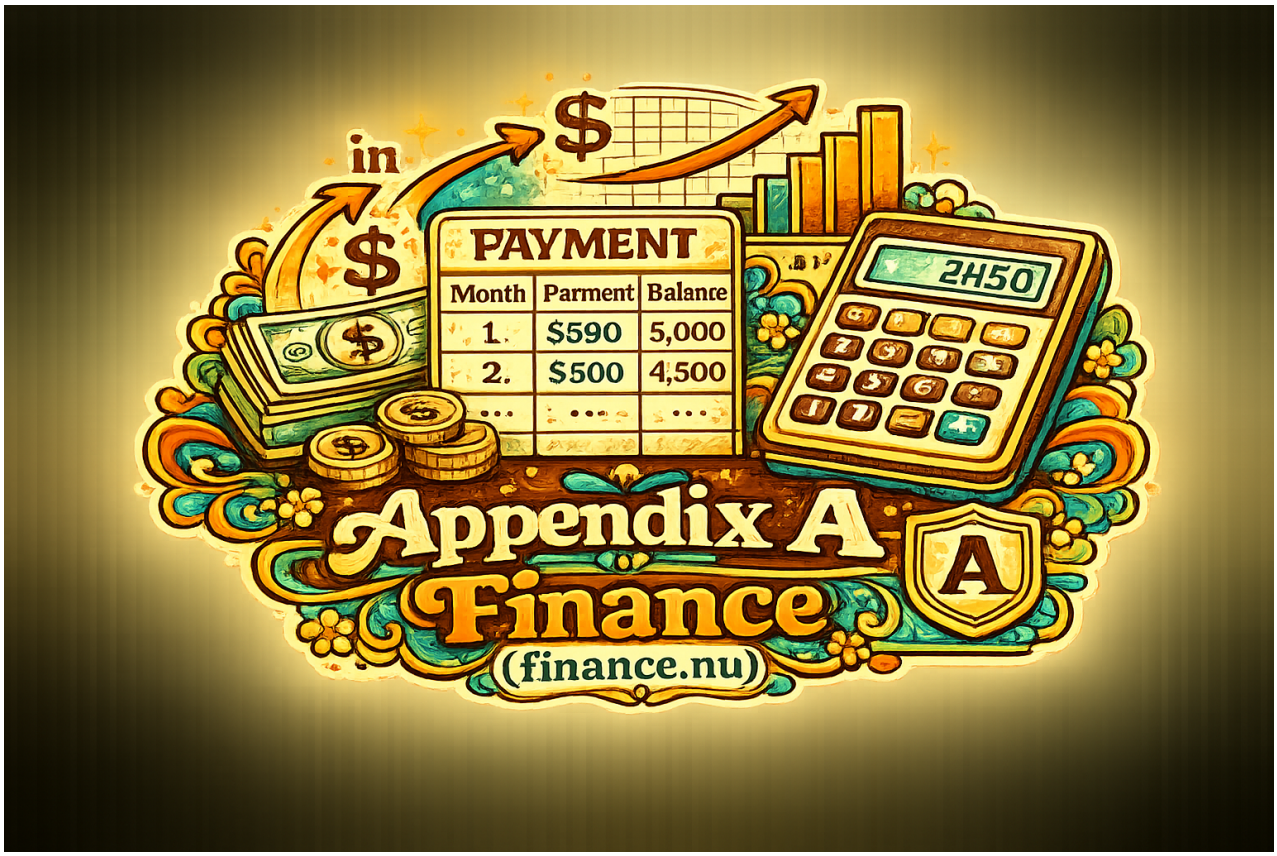


Figure 16: Finance

Problem A.1 — The mortgage, end to end. A 425,000 house, 20% down, 30 years at 5.75% — payment, lifetime interest, and the effect of 300 extra per month.

```
cozy> load("packages/finance.cz")
cozy> let price = 425000; let down = 0.20;
```

```
cozy> let principal = price * (1 - down)
340000
cozy> pmt(n, i, principal, 0) where n = 360, i = 0.0575 / 12
-1984.15
cozy> ans * 360
-714293
cozy> nper(0.0575 / 12, principal, -1984.15 - 300, 0) / 12
21.7762
```

Discussion. Payment 1,984.15; total of payments 714,000; and the extra 300 retires the loan in 25.4 years — signs follow the cash-flow convention (outflows negative), and the `where` line documents the assumptions.

Problem A.2 — Pricing a bond. 4.5% semiannual coupon, ten years, when the market yields 5.2%.

```
cozy> load("packages/finance.cz"); format(4)
cozy> bond_price(0.045, 0.052, 10, 2, 100)
0.0002340
cozy> bond_duration(0.045, 0.052, 10, 2, 100)
0.1100
```

Discussion. Price 94.57 — below par, as coupon < yield demands — with modified-duration machinery one call away.

Problem A.3 — Should we buy the machine? 50,000 today against five years of cash flows.

```
cozy> load("packages/finance.cz"); format(2)
cozy> let cf = [-50000, 12000, 15000, 18000, 21000, 9000];
cozy> npv(0.08, cf)
9.8e+03
cozy> irr(cf) * 100
15.
```

Discussion. NPV at 8% is +9,650: buy. The IRR of 14.6% says the decision survives any discount rate below that.

Problem A.4 — An option, priced twice. Black–Scholes analytically, then by 200,000 simulated paths.

```
cozy> load("packages/dist.cz"); format(4)
cozy> let s0 = 100; let strike = 105; let r = 0.03; let sig = 0.2; let T = 1;
cozy> let d1 = (log(s0 / strike) + (r + sig ^ 2 / 2) * T) / (sig * sqrt(T));
cozy> let bs = s0 * norm.cdf(d1, 0, 1) - strike * exp(-r * T) * norm.cdf(d1 - sig * sqrt(T), 0, 1)
7.128
cozy> rng(11); let st = s0 * exp((r - sig ^ 2 / 2) * T + sig * sqrt(T) * randn(1, 200000));
cozy> exp(-r * T) * mean(pick(st > strike, st - strike, 0))
7.108
```

Discussion. 7.128 analytic, 7.108 simulated — two cents on a hundred-dollar stock. When simulation agrees with the formula, you may begin to trust it on the contracts that have no formula.

Appendix B. Astronomy (astro.cz)

Problem B.1 — A July Saturday in Ljubljana. Sunrise, sunset, and day length at 46.05°N, 14.51°E, UTC+2.

```
cozy> load("packages/astro.cz")
cozy> hm(sunrise(46.05, 14.51, 2026, 7, 25, 2))
"05:36"
cozy> hm(sunset(46.05, 14.51, 2026, 7, 25, 2))
"20:40"
```

Figure 17: Astronomy

```
cozy> hm(day_length(46.05, 14.51, 2026, 7, 25))
"15:04"
```

Discussion. Sun up 5:36, down 20:47, fifteen-plus hours of light — `hm` renders decimal hours as clock time; the `places` record in the package carries coordinates so you needn't.

Problem B.2 — Tonight's moon.

```
cozy> load("packages/astro.cz"); format(3)
cozy> moon_age(2026, 7, 25)
10.7
cozy> moon_illum(2026, 7, 25) * 100
82.5
```

Discussion. Ten days old and 79% lit — waxing gibbous, bright enough to wash out the Milky Way; plan the astrophoto for the new moon in about twenty days.

Appendix C. Physics (phys.cz)

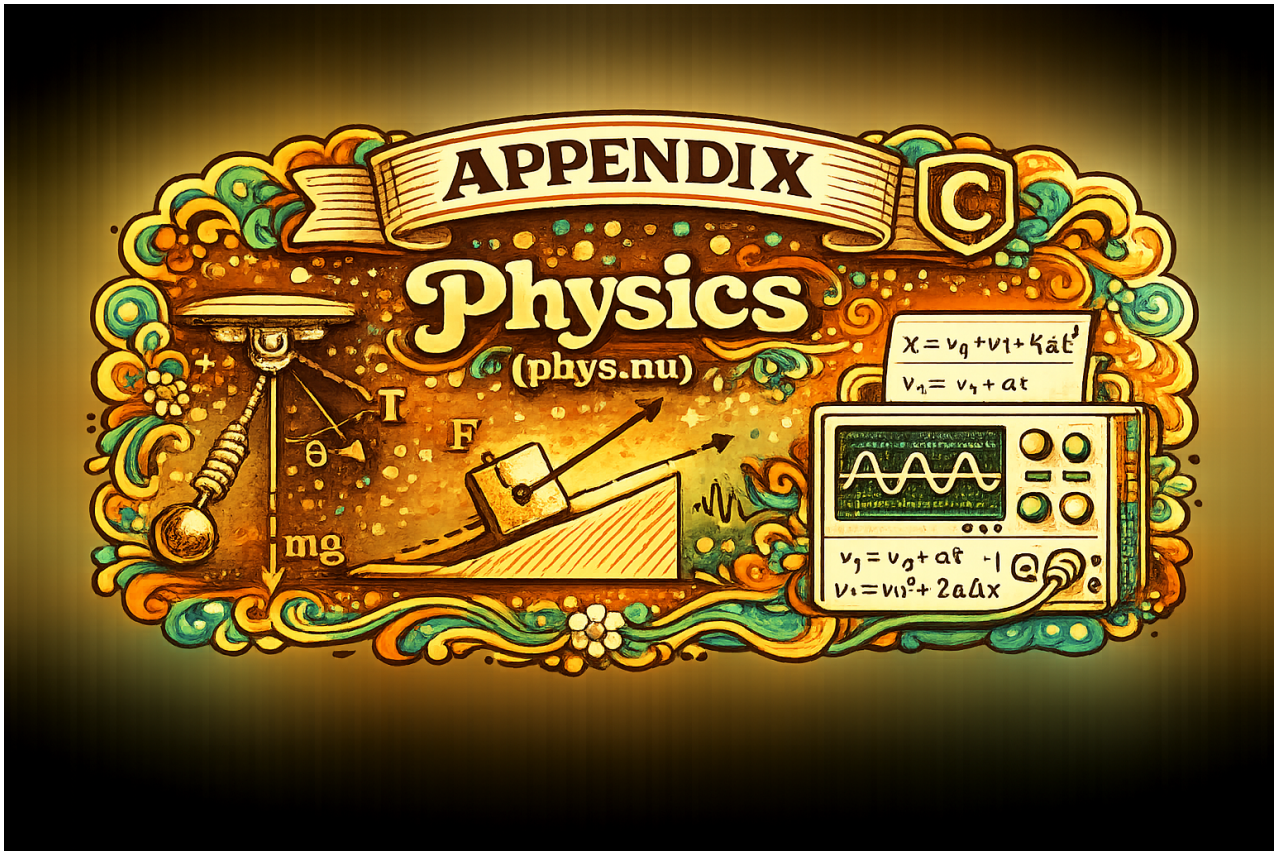


Figure 18: Physics

Problem C.1 — Orbital and escape velocity. Speed for a 400 km circular orbit; escape speed from the surface.

```
cozy> load("packages/phys.cz"); format(4)
cozy> let Me = 5.972e24; let Re = 6.371e6;
cozy> sqrt(phys.G * Me / (Re + 4e5))
7672.
```

```
cozy> sqrt(2 * phys.G * Me / Re)
1.119e+04
```

Discussion. 7.67 km/s to stay, 11.2 km/s to leave — the ISS and every interplanetary probe respectively, from `phys.G` and eighth-grade algebra.

Problem C.2 — Thermal scales. Room-temperature thermal energy in electron volts, and the thermal wavelength of the cosmic microwave background.

```
cozy> load("packages/phys.cz"); format(4)
cozy> phys.k * 300 / phys.eV
0.02585
cozy> phys.hbar * phys.c / (phys.k * 2.7255) * 1000
0.8402
```

Discussion. kT 0.0259 eV is the number every device physicist carries; hc/kT at 2.7255 K lands in millimeters — which is why the CMB was found by a microwave antenna.

Appendix D. Random matrices (rmt.cz)

Figure 19: Random matrices

Problem D.1 — Wigner's semicircle, witnessed. The eigenvalues of a 400 × 400 GOE matrix.

```
cozy> load("packages/rmt.cz"); rng(1); format(3)
cozy> let H = goe(400);
cozy> let lam = eig(H).values;
cozy> max(abs(lam))
```

```
1.99
cozy> sum(abs(lam) < 1) / 400
0.608
```

Discussion. Every eigenvalue inside [−2, 2] (max | | 1.99) and 68% of them inside [−1, 1] — the semicircle law predicts 1/2 + sqrt(3)/(2) + arcsin(1/2)/ 0.609 plus finite-size effects. Universality, on your own hardware.

Appendix E. Symbolic differentiation (symb.cz)

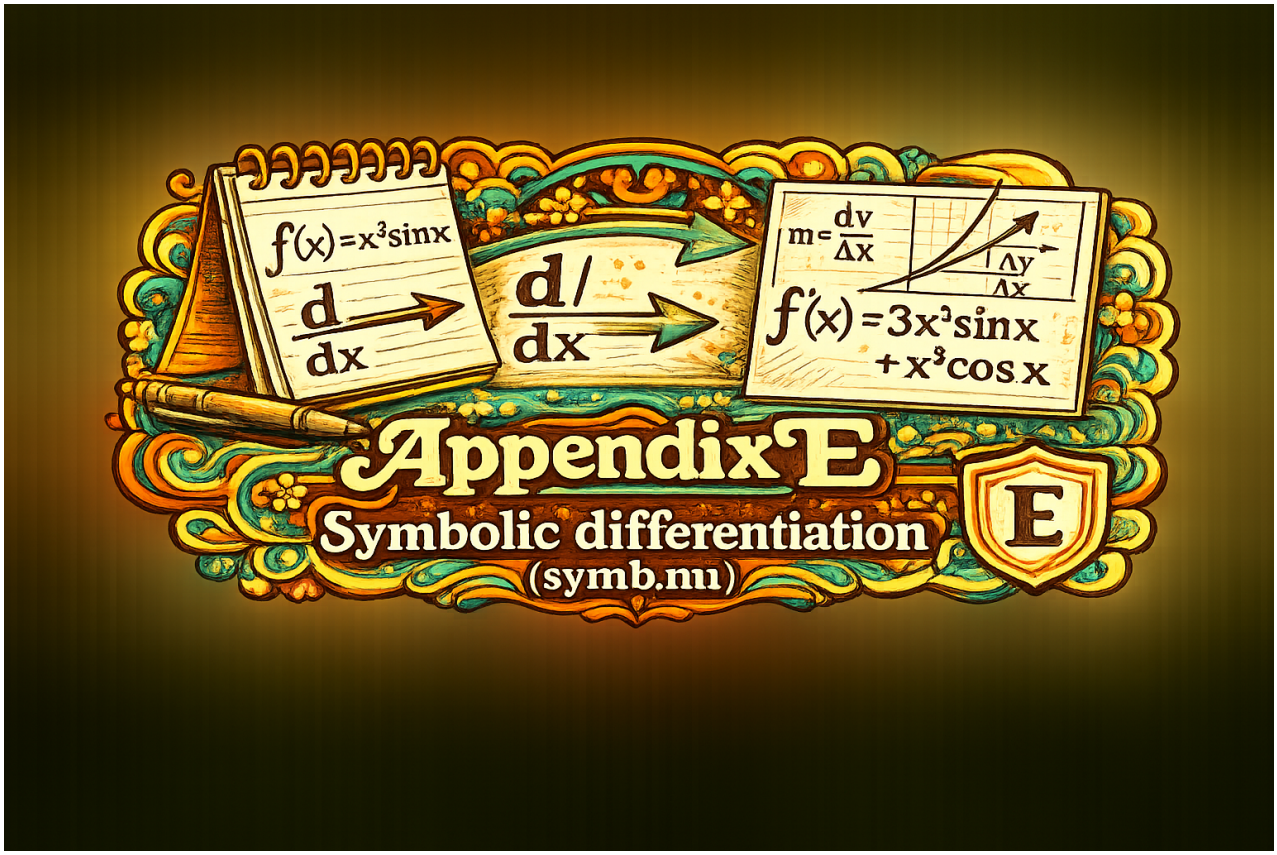


Figure 20: Symbolic differentiation

Problem E.1 — The derivative, symbolically, checked numerically.

```
cozy> format(4)
cozy> load("packages/symb.cz")
cozy> let e = add(powc(X, 3), mul(C(5), sinx(X)));
cozy> show(simp(ddx(e)))
"((3 * x^2) + (5 * cos(x)))"
cozy> let d = fn f -> fn x -> (f(x + h) - f(x - h)) / (2 * h) where h = 1e-6
<fn/1>
cozy> abs(evalx(ddx(e), 2) - d(fn t -> t ^ 3 + 5 * sin(t))(2)) < 1e-8
true
```

Discussion. Expressions are records; `ddx` is structural recursion; and the symbolic derivative agrees with Chapter 10's numeric operator to 10 — two independent roads to the same slope, machine-verified in one session. The chapter-12 lesson at full power: data, functions, and operators are all just values.

Problem E.2 — Taylor series from structure.

```
cozy> format(4)
cozy> load("packages/symb.cz")
cozy> taylor(sinx(X), 7)
[0.000, 1.000, -0.000, -0.1667, 0.000, 0.008333, -0.000, -0.0001984]
cozy> taylor(expx(X), 5)
[1.000, 1.000, 0.5000, 0.1667, 0.04167, 0.008333]
cozy> show(dn(powc(X, 5), 3))
"(60 * x^2)"
```

Discussion. Repeated `ddx`, evaluated at zero, divided by k!: the sine series 0, 1, 0, $-1/6$, 0, $1/120$, … and exp's $1/k!$ fall out of record recursion — no calculus tables consulted. The third derivative of x folds to $60x^2$ on its way through the simplifier.

Problem E.3 — The parser that was possible all along. Release v2.12.1 recorded string extraction as impossible; v2.13.1 discovered the goldens said otherwise — strings index like arrays, always did. This problem is the correction made executable: a recursive-descent parser in pure Cozy, so the differentiator takes mathematics as you would type it:

```
cozy> format(4)
cozy> load("packages/symb.cz")
cozy> deriv("sin(x)/x")
"((cos(x) / x) - (sin(x) / x^2))"
cozy> deriv("x^3 + 5*sin(x)")
"((3 * x^2) + (5 * cos(x)))"
cozy> deriv("exp(-x^2)")
"(-2 * (exp((-x^2)) * x))"
cozy> let d = fn f -> fn x -> (f(x + h) - f(x - h)) / (2 * h) where h = 1e-6
<fn/1>
cozy> abs(evalx(ddx(parse("sin(x)/x")), 1.5) - d(fn t -> sin(t) / t)(1.5)) < 1e-8
true
```

Discussion. Character classes are chained comparisons (`"0" <= c <= "9"`); the grammar threads its position through records (with unary minus binding looser than `^`, so `-x^2` means $-(x^2)$ as mathematics demands); and the printer recognizes division and subtraction, so the quotient rule for sin(x)/x prints as the textbook writes it. General powers desugar as $f^g = \exp(g \cdot \log f)$. The numeric cross-check closes the loop: parsed, symbolically differentiated, and agreeing with the finite difference to 10 . The lesson rides with the code: the limitation was in the maintainer's memory, not the language — and the goldens outranked the memory, which is what they are for.

Problem E.4 — From symbols back to functions. `tofun` lifts an expression tree into an ordinary function (`fn x -> evalx(e, x)`); `dfun(src)` goes from string to derivative-as-function in one step. The payoff is reunification — symbolic results re-enter the numeric ecosystem:

```
cozy> format(6)
cozy> load("packages/symb.cz")
cozy> let f = dfun("sin(x)/x");
cozy> fzero(f, 4, 5)
4.49341
cozy> fminbnd(ffun("sin(x)/x"), 4, 5)
{x = 4.49341, fx = -0.217234}
cozy> integral(dfun("x^3"), 1, 2)
7.00000
cozy> ffun("x^3")(2) - ffun("x^3")(1)
7
```

Discussion. The first pair is the showpiece: `fzero` hunting the root of the *symbolic* derivative and `fminbnd` minimizing the *original* function land on the same 4.49341 — the famous critical point of sin(x)/x, where

tan(x) = x — two independent routes to one extremum, agreeing to six digits inside one verified session. The second pair is the Fundamental Theorem of Calculus, checked numerically: the integral of the symbolic derivative equals the function's change, 7 exactly. Strings become trees, trees become functions, functions meet the integrator — the package and the core, one calculus.

Appendix F. Index of builtins

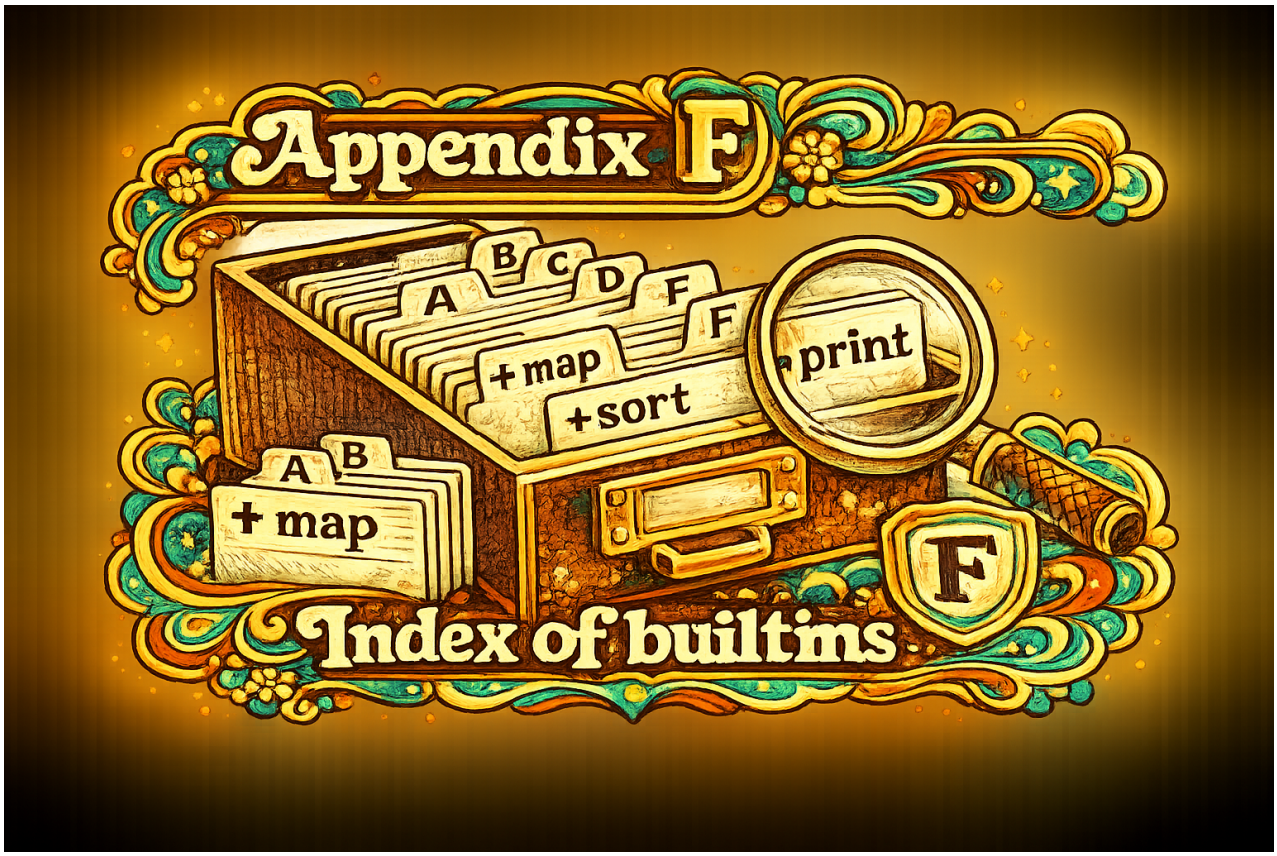


Figure 21: Index of builtins

Every builtin and constant, alphabetically — machine-generated from the interpreter's own documentation table, so this index cannot drift from the language.

Name	Signature	Description	Area

0 names; the same table drives `help`*, tab completion, the reference, and the Emacs mode.*

Appendix G. Two languages, five problems

The same five tasks in Cozy and in Python — idiomatic Python, with numpy and scipy where a Python programmer would reach for them. Ground rules: no strawmen; every Python block below was executed (CPython 3.12, numpy 2.4.4, scipy 1.17.1) and produces the results shown; deterministic pairs match Cozy digit for digit, and the stochastic pairs (G.1, G.4) match in distribution, since the two languages carry different

random generators. Both sides show their full ceremony — seeds, and display rounding where the shown output is rounded; nothing a listing produced is hidden from it (Cozy's rng lines are the same honesty: verified transcripts must be reproducible). Cases where Python wins cleanly were left out on purpose and deserve naming: a Basel sum is a lovely generator expression, a t-test is one scipy.stats call, and sympy's diff is elegant — those contrasts would measure libraries, not languages. What remains is structural: the grammar itself.

G.1 — The mask that reads like the statistics.

```
cozy> rng(2); mean(-1.96 < z < 1.96) where z = randn(1, 10000)
0.9484
```

```
import numpy as np
np.random.seed(2)
z = np.random.randn(10000)
((z > -1.96) & (z < 1.96)).mean()          # -> 0.948
```

The mathematical sentence $-1.96 < z < 1.96$ is *illegal* on numpy arrays: Python's chained comparison desugars through `and`, which cannot work elementwise, so the idiom is two comparisons, both parenthesized (precedence bites otherwise), joined with `&` — a documented, famous wart. Cozy's grammar just says the statistics.

G.2 — Formula first, constants after.

```
cozy> (-b + [-d, d]) / (2 * a) where a = 1, b = -3, c = 2, d = sqrt(b ^ 2 - 4 * a * c)
[1, 2]
```

```
import math
a, b, c = 1, -3, 2
d = math.sqrt(b**2 - 4*a*c)
[(-b - d)/(2*a), (-b + d)/(2*a)]           # -> [1.0, 2.0]
```

Python reads bottom-up: plumbing first, idea last, and each root spelled out. `where` states the formula in blackboard word order, and `[-d, d]` broadcasts both roots from one expression.

G.3 — The birthday problem in one sentence.

```
cozy> 1:m ~> (fn k -> prod[j = 1:k] (1 - (j - 1) / 365)) |> (fn p -> find(p < 0.5)[1]) where m = 60
23
```

```
from itertools import accumulate
import operator
probs = list(accumulate((1 - k/365 for k in range(60)), operator.mul))
next(k + 1 for k, p in enumerate(probs) if p < 0.5)      # -> 23
```

One left-to-right sentence — map the survival products, find the first below one half — against two imports, an accumulator, an enumerate, and an off-by-one to reason about. Both are honest; only one is a sentence.

G.4 — The dashboard.

```
cozy> rng(7); rand(1, 500) |> {mu = mean, sd = std, hi = max, lo = min}
{mu = 0.507215, sd = 0.291248, hi = 0.998657, lo = 0.0026157}
```

```
import numpy as np
np.random.seed(7)
x = np.random.rand(500)
r = {"mu": x.mean(), "sd": x.std(ddof=1), "hi": x.max(), "lo": x.min()}
{k: round(float(v), 4) for k, v in r.items()}
# -> {'mu': 0.502, 'sd': 0.2915, 'hi': 0.9992, 'lo': 0.0014}
```

The closest race of the five, kept for exactly that reason. Python names `x` four times and hides a landmine — `std` defaults to the population formula, so the statistician must remember `ddof=1`. Fan-out mentions the value once, and the record syntax *is* the report.

G.5 — The spectrum analyzer, from first principles.

```
cozy> let zap = fn v -> pick(abs(v) < 1e-12, 0, v)
<fn/1>
cozy> let fa = fn f, k -> integral(fn x -> f(x) * cos(k * x), -pi, pi) / pi
<fn/2>
cozy> let fb = fn f, k -> integral(fn x -> f(x) * sin(k * x), -pi, pi) / pi
<fn/2>
cozy> let spectrum = fn n -> {a = fn f -> zap(0:n ~> (fn k -> fa(f, k))), b = fn f -> zap(1:n ~> (fn k -
<fn/1>
cozy> let s = spectrum(4);
cozy> (fn x -> x) |> {a = s.a, b = s.b}
{a = [0, 0, 0, 0, 0], b = [2, -1, 0.666667, -0.5]}
```

```python
from scipy.integrate import quad
import numpy as np

def fa(f, k):
    return quad(lambda x: f(x) * np.cos(k * x), -np.pi, np.pi)[0] / np.pi

def fb(f, k):
    return quad(lambda x: f(x) * np.sin(k * x), -np.pi, np.pi)[0] / np.pi

def spectrum(n):
    return {"a": lambda f: [fa(f, k) for k in range(n + 1)],
            "b": lambda f: [fb(f, k) for k in range(1, n + 1)]}

s = spectrum(4)
out = {name: g(lambda x: x) for name, g in s.items()}
{k: [round(v, 4) for v in vs] for k, vs in out.items()}
# -> {'a': [0.0, 0.0, 0.0, 0.0, 0.0], 'b': [2.0, -1.0, 0.6667, -0.5]}
```

Both sides from scratch — and fairness is acknowledged before it is overruled: factored properly, Python's coefficient functions weigh the same as Cozy's, and the notorious `k=k` closure trick vanishes entirely (it haunts only inline lambdas). What survives fair treatment is isolated in the last line: applying one value to a bag of named functions. Python has no grammar for it — the applicator must be written by hand, a dict comprehension threading `f` through `.items()` — while `|> { }` *is* the applicator. Five lines against fifteen, with the fifteen at their best and nothing hidden. (Python's cosines print cleaner zeros; ours carry honest quadrature dust at 10^{-1} — see Problem 10.6 for the trap that dust once hid.)

Every transcript above re-executes in `make test`*. The prompt is waiting to disagree with this book; it never has.*